SKETCHPAD HELP

Overview: Tabs, Subsystems, Design Notes

Checked against the code after Phase 8, with three new pages

Built 27 October 2026 from claude/docs at 4392c1d.
31 pages, from `help/Overview`.

Contents

BEFORE THE PAGES

About this collection 4
Things found while checking 5

TABS

Figures Tab — A First Discussion NEW 6
Home Tab — How It Works 10
Draw Tab — How It Works 12
Patterns Tab — How It Works 15
Gallery Tab — How It Works 17
Utilities Tab — How It Works 19

SUBSYSTEMS

setUI.js — Switching Tabs 21
uiState — The App's Memory 23
uiUtilities.js — Shared Screen Helpers 25
The Manifest System: Loading, Caching, Invalidation, and Tab Registration 27
fileLayer.js — Reading Files and Building Paths 34
nodeLayer.js (Browser → Node Dispatch Bridge) 36
categories.js — Category Rendering System (Description) 41
caption.js — The Caption Bar 44
menuManager.js — The Caption Menu 45
overlay.js — Help, Notes and Panel Overlays 47
interactor.js — Handles You Can Drag 49
Point Dragging NEW 51
Parameter Controls — Sliders and Their Kin 54
parameterControls.js — Control Schema Flags & Rendering Rules 56
Control Grouping System - Implementation Guide 60
scriptRunner.js (Universal Script Loader + Execution Pipeline) 64
actionsRegistry.js — Caption Menu Actions Registry (Description) 68
ui_callbacks.js — Retired 70

DESIGN NOTES

Sketchpad Architecture (Detailed) 71
Sketchpad Data Flow (Overview) 85
How Sketchpad drawRegistry Files Are Organized and Used 91
runPattern() in Sketchpad 99
Design Rules: Gallery Script Conversion (Authoritative) 104

Coding Simplicity 109
Undo in Figures: a Proposal NEW 114

About this collection

This is the `help/Overview` section of Sketchpad's help, all pages in one document. In the app the same pages open from the Help browser (Commands → Help).

What changed on 27 October 2026. Every page was read against the code as it stands after Phase 8 (welds). Each page now opens with a yellow note saying when it was checked and what changed:

- **Rewritten** where the code had moved on: the five tab pages (each tab is now several files), and most subsystem pages (handles, tab switching, overlays, menus, parameter controls, file paths).
- **Corrected in place** where the page was right in substance: the manifest system, the script runner, categories, the Node bridge, the design notes.
- **New** (marked NEW in the contents): a first discussion of the Figures tab with the phase list, a page on point dragging, and a proposal for undo in Figures.

Suggested reading. Start with *Figures Tab — A First Discussion*, then *Point Dragging*, then *Switching Tabs* and *uiState*. The design notes can be read in any order. A separate document, *The Figures code, commented*, has the Figures source files themselves.

Things found while checking

Checking the pages against the code turned up a few things in the code itself. None of them was changed (the documentation branch changes only comments and documents); they are listed here, and on the page concerned, for a decision.

Where	What	Kind
Figures: weld mark	The + is drawn under the handle dot, so a weld on a handle shows only a white ring.	Visible bug
Figures: centroid	A right-press on the diamond starts a move (the button is not checked).	Small bug
Utilities	Leaving while on Result and coming back re-runs the script but hides the Result subtab (read from the code, not tried).	Probable bug
`setUI.js`	No tab's `save()` is ever called; Draw, Utilities and Figures therefore always re-init (harmless: they return to where they were). End-of-work items 4 and 6 in `PHASES.md`.	Design loose end
`caption.js`	`buildMenuHandler` calls the menu with the wrong arguments; it and two other helpers are unused.	Dead code
`categories.js`	`buildCategoryDescriptor` passes a label function to `sort()`; it leaves the order unchanged, which Patterns happens to rely on.	Quirk
`scriptRunner.js`	Its header says `runPattern(ctx)` receives the context; the code never passes it.	Stale comment
`manifest.js`	A `drawRegistry` branch that nothing reaches and that would find nothing.	Dead code
Assorted	`actionRegistry.js`'s table, `clearHomeCanvasAndOverlays`, `rebuildAllSystemCaches`, `setCommandsButton`, `clearOverlaysForTabSwitch`, the fileLayer normalizers, two session-storage `rehydrate` functions: all exported, none called.	Dead code
Parameter controls	The point-picker widgets are switched off, but `boxes` and `regularPursuit` still declare one (it shows as a plain text box).	Leftover

Figures Tab — A First Discussion

NEW

← Contents

> Written 27 October 2026, after Phase 8 (welds) was merged into `claude1`. This is a preliminary account: what the tab is for, the ideas it is built on, how it is used today, how the code is laid out, and where the phases stand. The authority for the phases is `PHASES.md`.

What it is for

Figures is where several drawing objects come together into one design. It serves two purposes, and neither is a means to the other:

1. **A printable point template** for pounding nails into wood: every hole numbered and placed, at true size.
2. **A place to sketch**: to explore designs by working with how objects are represented and how they relate to each other. The original StringThing could not do this, and it is why the program is called Sketchpad.

When a decision trades one purpose against the other (trimming a control range, dropping an object that nails cannot use, forcing every object through one drawing path), it is raised for decision rather than settled quietly.

The ideas, in the order they build on each other

Figure
: One design, in its own subtab ("Untitled 1"). Plain data: name, layers, discs, welds. It lives in `uiState.figures.tabs[tabId].figure`, so it survives switching tabs. It is not saved to disk yet (Phase 10).

Layer
: One inserted registry object (a Line, a Parabola, a Regular Polygon…) with its parameters, a visibility flag, an identity color, and a transform. Layer numbers never change. Stacking order is kept separately; the last layer clicked goes on top.

Instance
: The live object that draws a layer: a private copy of the registry object, so two Parabolas do not share parameters. Kept beside the figure, not in it.

Transform
: How far a layer has been moved and turned: `{ angle, tx, ty }`. The object keeps its own numbers; only the transform changes. Every drawn line, handle and disc goes through it.

Handle
: One of the object's own red dots. The only thing that drags the object's shape. Arrow keys nudge the selected one (1 px, 10 with Shift).

Infrastructure

The skeleton of an object: the lines (arms) the stitches run between, as opposed to the stitches themselves. Each arm is divided into candidate points, the places a disc can sit.

Disc

A numbered point, drawn as a filled circle in its layer's color. *Anchored*: sits at a fixed fraction along an arm and rides with the object. *Free*: placed anywhere and moved only by dragging it. Numbers are never reused.

Centroid

The diamond at the center of the active layer's extent. Drag it to move the layer; click it for a box with a rotation handle.

Weld

A group of discs, on different layers (or two corners of one object), that must stay on the same spot. Drag one, and the others' objects follow.

The screen

- **Left column**: Commands; Show All (every layer at full strength, hidden ones too); Points Mode; Create Figure (on Categories) or Insert Object (in a figure).
- **Caption bar**: the figure's name; Line and Parabola buttons; "Layer N · Name" and its Show checkbox; the "v" menu (Help, Notes, Hit radius, Delete selected point, Break apart, and grayed future items).
- **Layer strip**, left of the canvas: one numbered chip per layer, in its color. Click to make it active; right-click to change its color. A hidden layer's number is struck through.
- **Canvas**: the layers, the discs, the weld marks; above it the centroid overlay and the handles.
- **Action panel**: the active layer's sliders, built by the same code the Draw tab uses.

How it is used today

Lines mode (the default)

- Insert objects. A second copy of the same object is offset 20 px so it does not land on the first.
- Drag handles to shape the active object. Drag the diamond to move it; click it and drag the round handle to turn it.
- Hide a layer while working (it still appears in Show All and the final drawing).
- Break apart: a composite object becomes separate layers, one per piece. The original is hidden.

Points mode

- Layers show only their points. Only the active layer can be changed, and the insert tools are grayed.
- Left-click near a candidate: an anchored disc. Elsewhere: a free disc. On a disc: select it. Drag a free disc to move it. Delete or Backspace removes the selected disc.
- Right-click on a spot where two discs meet: weld them. Right-click a welded disc: unweld it. A short note in the caption bar says what happened.

Making a weld, step by step

1. Points Mode on.
2. Make the first object's layer active and click the meeting spot: a disc on that object.
3. Make the second object's layer active and click the same spot: a disc on that object.
4. Right-click the spot. "Welded."
5. Points Mode off. Drag either object's handle there; the other follows.

The layer change between steps 2 and 3 is easy to forget, because a click only ever places a disc on the active layer. The next Phase 8 pass is meant to remove it: drop one handle on another and they weld.

The code

File	Owns
`figures.js`	the TabSpec; init; the left-column controls
`figuresNav.js`	subtabs: open, switch, close; wiring the checkboxes and Insert list
`figuresCommands.js`	Commands panel; the caption menu
`figuresDisplay.js`	the saved-figure category list (empty until Phase 10)
`figuresState.js`	navigation bookkeeping
`figureModel.js`	the figure as plain data
`figureLayers.js`	insert, draw, select, hide layers; arming the mouse handling
`figurePoints.js`	discs: data, finding candidates, drawing
`figurePointer.js`	mouse and keys in points mode
`figureCentroid.js`	moving and turning a whole layer
`figureWelds.js`	welds: make, unmake, follow, draw the mark
`draw/drawTransform.js`	the transform math

A separate document, *The Figures code, commented* (`docs/Figures-code.pdf`), has every one of these files in full, with comments for reading and two actions traced end to end.

Shared pieces Figures relies on: `interactor.js` (handle dragging; see "Point Dragging"), `drawLine` in `draw/drawUtilities.js` (every line goes through it, which is how the transform reaches every object), `buildParameterControls()`, and each registry object's `infrastructure()` and, for Break apart, `decompose()`.

Where the phases stand

Phase	What	Status
1–4	Tests and safe fixes; hygiene; Notes popup; Draw prerequisites (final flag, `drawLine` routing)	Done
5	Figures shell: Create Figure, Untitled N subtabs, caption menu	Done

6a	Insert objects; layer strip; Show All	Done
6b	Points mode: anchored and free discs, hit radius	Done
6c	Layer colors; unnumbered colored discs; free discs draggable	Done
7	Arrow-key nudging of handles	Done
7b	Centroid, bounding box, move and rotate a layer	Done
7c	Break apart	Done
7d	`infrastructure()` for every array-built object	Done
8	Welds; two real-use bug fixes on 27 October	Done, merged into `claude1` 27 Oct 2026
8, next pass	A handle dropped on another handle welds them	Specified; six questions open in `PHASES.md`
8b	Magic points: midpoints, tops of loops, cusps as candidates	Not started
9	Print aids: scale bar, minimum nail spacing	Not started
10	Save and recovery: Save As, categories, reload	Not started
11	Figure follow-ups (deleting layers among them)	Not started
12	Thumbnails in the Insert list	Not started
13	Sweep tool	Not started
14	Filesystem boundary refactor	Not started

Known gaps today

- Nothing is saved: closing a figure subtab or reloading the page loses it (Phase 10). There is no undo (see "Undo in Figures: a proposal" under Design Notes).
- The weld mark (a +) is hidden under the handle dot whenever a weld sits on a handle, which is nearly always; you see only its white ring.
- A right-press on the centroid diamond (lines mode) starts a move; the same right-button fix made to handles and points mode has not been made there.
- A weld moves its partners by shifting them. Turning a welded layer turns that layer; its partners are pulled along, not turned.
- Objects without `infrastructure()` take only free discs.

Home Tab — How It Works

← Contents

> Rewritten 27 October 2026 against the current code. The earlier page described a single `home.js`; the tab is now six files under `ui/home/`. Two things changed in behavior: images now show in `#text` (not `#sketchpad`), and the Home manifest now lives in the shared `manifest.cache` instead of a private cache.

Home is a dispatcher. It reads one flat list, `/home/manifest.json`, groups the entries by their `status`, and shows one category frame per status. Clicking an entry either shows it here (a script on the canvas, or an image) or, for a built-in Draw object, opens the Draw tab on it.

The files

File	Owns
`home.js`	`HomeTabSpec`; init / restore / save; `ensureHomeSavedState()`
`homeNav.js`	the two subtabs, switching between Categories and Results, the category frames and what a click does, launching Draw
`homeManifest.js`	loading `/home/manifest.json`, grouping by status, refreshing after an edit
`homeResults.js`	showing the chosen entry: run a script or load an image; the render token
`homeMenuCmds.js`	caption bar with the file's full path; menu (Help, Show Script, Edit Manifest); the Commands panel
`homeState.js`	the grouped manifest and the render token

State

```
uiState.home.saved = {
  view:         "categories" | "results",
  activeStatus: string | null,
  activeIndex:  number | null,
  activeEntry:  the manifest entry object | null
}
```

Unlike the other tabs, Home's `saved` is already filled in by `uiState.js`, so `setUI.js` always calls `restore()` for Home (after a manifest rebuild it calls `init(true)` instead). Both end in `switchHomeView(saved.view)`, so Home always comes back to what it last showed. `saveHomeState()` exists but is never called; the state is written as you click.

Two views

- **Categories**: clears the caption, the canvas area and `#action`, then draws the status frames into `#text` (or "loading..." until the manifest arrives).
- **Results**: shown only once an entry has been chosen (the Results subtab appears when `activeEntry` is set). Runs `renderHomeResults()`.

The subtab bar is built by the shared `buildSubtabBar()` helper from a small description (an "adapter"): the two tabs, which one is active, and a `condition` that hides Results until there is something to show.

What a click does

- An entry with `sourceType: "drawRegistry"` is launch-only. Home resets its own view to Categories, writes a launch intent into `uiState.launch` (pending, from home, to draw, the registry key), and calls `setUI("draw")` through a dynamic `import()` (`setUI.js` imports Home, so Home cannot import it at load time). Draw consumes and clears the intent.
- Any other entry is stored as `activeEntry` and shown in Results.

Results

- A `.js` path is run with `runScriptByPath()` (`scriptRunner.js`) on the shared canvas in `#sketchpad`, with its parameter controls in `#action`.
- An image path (png, jpg, jpeg) is loaded and shown in `#text`, at most 800 by 800.
- Anything else shows an error and throws.

The render token. Each Results render takes a new number from `incrementHomeResultsRenderToken()`. A slow image load checks, when it finishes, that its number is still the latest; if you clicked something else meanwhile, it quietly gives up instead of overwriting the newer result.

The manifest

`/home/manifest.json` is a flat array (no category folders, no directory registry). It is read with `fileLayer.loadJSON()` and kept in `manifest.cache["home"]`, so the shared cache invalidation clears it like any other. Entries whose `status` is missing or blank are skipped; a non-object entry is an error.

Caption and commands

In Results the caption shows the title and, just before the menu button, the entry's full path. The menu has Help, Show Script and Edit Manifest. The Commands panel runs Rebuild & Validate, resyncs, re-runs `initHomeTab()`, and prints the report; it also has a Help button.

Leftovers

- `clearHomeCanvasAndOverlays()` in `homeResults.js` is exported but no longer called; canvas clearing is done by the script runner.

Draw Tab — How It Works

← Contents

> Rewritten 27 October 2026 against the current code. The earlier page described a single `draw.js`; the tab has since been split into seven files under `ui/draw/`, and it gained secondary objects (saved parameter variations).

The Draw tab shows one registry object at a time on the shared canvas, with its controls. It is a small tab system inside the main one: a permanent **Categories** subtab that lists every `drawRegistry` object, plus one subtab per object you open.

The files

File	Owns
`draw.js`	`DrawTabSpec`, `DrawController`, init / restore / save, and the launch intent from Home
`drawNav.js`	the subtab bar: open, switch, close; the dirty mark; secondary objects (find, list, load)
`drawCategories.js`	the Categories view: read `window.drawRegistry`, group by category, render with `categories.js`
`drawRunner.js`	`drawActiveTab()`: put the canvas in `#sketchpad`, build the controls, `update()` then `draw()`; scaffold guides; export to a Pattern script
`drawMenuCmds.js`	caption bar and its menu; Save / Copy / Reset; PNG and Pattern export; secondary save and archive; the Commands panel
`drawTabState.js`	one shared value: the set of registry ids that have secondaries on disk
`secondaryObjects.js`	the requests to the Node service that save, list, load and archive secondaries

State

```
uiState.draw = {
  activeSubtab: "tab-categories" | "tab-<object name>",
  tabs: {
    "tab-categories": { type: "categories" },
    "tab-parabola":   { type: "object", drawRegistry: <the registry entry>,
                        dirty, parameters, showControls,
                        redrawHandler, onParamChange }
  },
  saved: null
}
```

An object subtab holds the *registry entry itself*, not a copy (contrast Figures, where each layer gets its own instance). Its `parameters` is the entry's own `params` object. `redrawHandler` and `onParamChange` are attached by `drawActiveTab()` for the parameter controls to call.

Activation: init, and why restore never runs

`setUI.js` decides how to show a tab: `init(true)` if the tab is flagged `needsUpdate`; otherwise `restore()` if `uiState.draw.saved` is set; otherwise `init(false)`. Nothing ever sets `uiState.draw.saved` (setUI does not call `save()` on the way out), so Draw always runs `initDrawTab()`. That is fine in practice: `initDrawTab()` rebuilds the subtab bar from `uiState.draw.tabs` and switches back to the active subtab, so open objects survive a trip to another tab. `restoreDrawTab()` and `saveDrawState()` exist but are not reached.

`initDrawTab()` in order:

1. Starts finding which objects have secondaries (asynchronous; redraws the category list when done).
2. Clears the regions, labels and wires the Commands button.
3. Rebuilds the subtab bar from `uiState.draw.tabs` (or creates Categories if there are none).
4. If Home left a launch intent for Draw (`uiState.launch`), consumes and clears it and opens that object in a new subtab.
5. Otherwise switches to the remembered subtab, or Categories.

Opening, switching, closing

- **addDrawSubtab(item)** makes the subtab id from the object's name (`"tab-" + name, spaces to dashes, lower case`), calls the entry's `init()`, arms the interactor if the object is interactive and has handles, records the tab, and draws it.
- **switchTab(tabId)** marks the button active, disarms the interactor, clears the regions, and shows Categories or the object (draw, caption, re-arm the interactor).
- **deleteTab(tabId)** removes the button and the state and switches to a neighbor.

Drawing: drawActiveTab()

1. Show `#sketchpad-wrapper`; put the shared canvas into `#sketchpad`; clear it.
2. Size the "interaction" and "guides" overlay canvases to match.
3. Attach `redrawHandler` (this function) and `onParamChange` (mark dirty).
4. Build the controls with `buildParameterControls(state, "tab-draw", true)`.
5. `entry.update(params)`, `entry.draw()`, then the scaffold guide (a dashed gray line along `elements.l1`, for objects that have one).

Caption menu and commands

The caption shows the object's name and a "v" menu built each time it opens: Help, Notes, Save as PNG, Save as Pattern, Copy Object, Reset Parameters, then either Save as Secondary (for a primary object) or Save Secondary and Archive Secondary (for a loaded secondary). The Commands button opens a maintenance panel (Rebuild & Validate through the Node service).

Secondary objects

A secondary is a saved set of parameter values for one registry object, kept on disk by the Node service (`secondaryObjects.js`). On the Categories view, an object that has secondaries shows a ">"

button that lists them; choosing one loads its values into a subtab. `drawTabState.js` holds the set of ids that have any, filled by `updateSecondariesDiscovery()`.

Known gaps (still true on 27 October 2026)

- **Two subtabs of one object collide.** The id comes from the name, and the subtab holds the shared registry entry, so opening the same object twice gives one id and one set of parameters.
- **Restore is never used** (above). If whole-page reload persistence is wanted, it needs a real save call and a restore path that rebuilds subtabs from saved keys and parameters.
- Handles are drawn only for objects marked `interactive` with a `params.points` array.

Patterns Tab — How It Works

← Contents

> Rewritten 27 October 2026 against the current code. The earlier page described a single `patterns.js`; the tab is now five files under `ui/patterns/`. Several of the old page's "discussion points" have been settled: script paths are rooted (`/patterns/<category>/<file>.js`), scripts run through the shared script runner, and the canvas is attached in one place.

Patterns browses the saved pattern scripts under `/patterns/`, one category frame per folder. Choosing one runs it on the shared canvas, with a thumbnail strip of its category and Prev/Next.

The files

File	Owns
`patterns.js`	`PatternsTabSpec`, `PatternsController`; init / restore / save; loading the manifest
`patternsNav.js`	the subtab bar (Categories, and Pattern once one is chosen); the category view
`patternsDisplay.js`	running the chosen pattern, the thumbnail grid, Prev/Next
`patternsMenuCmds.js`	caption and menu (Help, Show Script, Notes, Create Thumbnail, Edit Manifest, Archive); the Commands panel
`patternsState.js`	the current category and index

State

```
uiState.patterns = {
  activeCategory, activeItem,
  saved: { view: "categories" | "pattern", activeCategory, activeItem }
}
```

The rule the code follows: any action that changes what you are looking at writes `uiState.patterns.saved` first, then draws. Because `saved` is set during navigation, `setUI.js` uses `restore()` when you come back to the tab, and the tab returns to the same view. (`setUI` never calls `save()`; it is not needed.)

`showSelectedPattern()` also writes `saved` to the browser's session storage, and `rehydratePatternsState()` can read it back, but nothing calls that function at present, so a page reload still starts at Categories.

Two views

- **Categories**: one frame per category, from `manifest.getCategoryMap("patterns")`. A click writes the new `saved` and shows the pattern.
- **Pattern**: the pattern on the canvas in `#sketchpad`, its category's thumbnails in `#action` (the current one highlighted), and a caption with the title, Prev/Next and the menu.

Showing a pattern: showSelectedPattern(category, index)

1. Write the module state, `uiState.patterns` and `saved` (and session storage).
2. Clear `#text`, `#action`, `#sketchpad`.
3. Run `/patterns/<category>/<filename>.js` with `runScriptByPath()` into `#sketchpad`, with controls enabled. An error is shown in red in its place.
4. Draw the thumbnail grid, the caption, and make sure the Pattern subtab exists and is active.

Prev and Next wrap around at the ends of the category.

The manifest

`ensurePatternsManifestLoaded()` asks the shared manifest system for "patterns" (`manifest.get`), which reads the category list and each category's `manifest.json` into `manifest.cache`. Everything else reads it through `manifest.getCategoryMap("patterns")`.

Commands panel

Rebuild & Validate asks the Node service to rebuild every manifest, clears the cache and marks every tab `needsUpdate` (`syncSystemStateAfterRebuild`), and redraws the current view from `saved`.

Small things worth knowing

- Prev/Next on an empty category quietly does nothing (it reads an empty list rather than failing).
- A restore that finds its category gone (after a rebuild) falls back to Categories; an index past the end is pulled back to the last item.

Gallery Tab — How It Works

← Contents

> Rewritten 27 October 2026 against the current code. The earlier page described a single `gallery.js` and listed risks found in a pasted copy; the tab is now five files under `ui/gallery/`, scripts run through the shared script runner, and the manifests come from the shared manifest system.

Gallery is a browser over three collections, or *domains*: **Ideabook** and **Patterns** (images) and **Scripts** (scripts that draw). Each has its own subtab of category frames. Choosing an item opens one shared Results subtab, labeled *Images* or *Drawings*, with Prev/Next.

The files

File	Owns
`gallery.js`	`GalleryTabSpec`, `GalleryController`; init / restore / save; loading the three domains' manifests
`galleryNav.js`	the three domain subtabs and the Results subtab; the category views
`galleryResults.js`	showing an image (with thumbnails) or running a script; Prev/Next; normalizing entry paths
`galleryMenuCmds.js`	caption ("category: title", Prev/Next, menu) and its commands (Help, Show Script, Notes, Edit Manifest, Archive); the Commands panel
`galleryState.js`	current domain, category, list and index

Where the content comes from

```
/gallery/<Domain>/directoryRegistry.json        the list of categories
/gallery/<Domain>/<Category>/manifest.json      the items in one category
/gallery/<Domain>/<Category>/images/thumb_<filename>.png
```

`ensureGalleryCacheLoaded()` asks the shared manifest system for `gallery/Ideabook`, `gallery/Patterns` and `gallery/Scripts`. `getGalleryCache()` returns the three category maps from `manifest.getCategoryMap()`. (`galleryState.js` still declares a `galleryCache` variable; the lookups no longer use it.)

State

```
uiState.gallery.saved = {
  view:     "categories" | "results",
  domain:   "Ideabook" | "Patterns" | "Scripts",
  category: string | null,
  index:    number | null
}
```

The tab rewrites `saved` every time you move, so `setUI.js` uses `restore()` when you return, and Gallery reopens exactly where it was. Gallery remembers one Results context only: the Results subtab always means "the last thing chosen". `saved` also goes to session storage, but `rehydrateGalleryState()`, which would read it back after a page reload, is not called.

The module state in `galleryState.js` (current list and index) drives Prev/Next. It is written together with `saved` by the same functions, so the two stay in step.

Results

- **Images** (Ideabook, Patterns): the image in `#text`, a thumbnail grid of the category in `#action` with the current one marked; clicking a thumbnail shows it.
- **Drawings** (Scripts): `/gallery/Scripts/<category>/<filename>` is run with `runScriptByPath()` into `#sketchpad`, controls enabled. No thumbnail grid.

Prev and Next move within the current category's list.

Restore details

- If the remembered category has vanished (after a rebuild), Gallery falls back to that domain's categories.
- An index past the end is pulled back to the last item.

Leftovers and shims

- `normalizeGalleryEntryPath()` strips a leading `Category/` from entry paths that include it. A manifest rule ("paths are relative to the category folder") is not enforced yet; this covers older entries.

Utilities Tab — How It Works

← Contents

> Rewritten 27 October 2026 against the current code. The tab is now five files under `ui/utilities/`, and scripts run through the shared script runner instead of being imported here.

Utilities runs small helper scripts. **Tools** scripts produce text or HTML, shown in `#text`. **Lab** scripts draw on the shared canvas. A third subtab, **Result**, appears after the first run and re-runs the last script.

The files

File	Owns
`utilities.js`	`UtilityTabSpec`; init / restore / save; loading the Tools and Lab manifests; refreshing after a manifest edit; the Commands panel
`utilitiesNav.js`	the Tools / Lab / Result subtabs and switching between them
`utilitiesDisplay.js`	the category frames, what a click does, running a script, showing a text result
`utilitiesMenuCmds.js`	caption ("category: title") and menu (Help, Show Script, Notes, Edit Manifest, Archive)
`utilitiesState.js`	current domain, category, list, index, and whether anything has run

Where the scripts are

```
/utilities/Tools/<Category>/<entry.path>
/utilities/Lab/<Category>/<entry.path>
```

Each group has a directory registry and per-category manifests, read through the shared manifest system (`utilities/Tools`, `utilities/Lab`).

Running a script: runUtilityEntry(subtab, category, entry)

- **Lab**: `runScriptByPath(path, "canvas", ...)` into `#sketchpad`, controls in `#action`.
- **Tools**: `runScriptByPath(path, "text", ...)`; a string result is kept in `uiState.utilities.lastResult` and shown in `#text`.
- Then the caption is set. An error is shown in `#text` and rethrown.

A click on an item records the subtab, category and entry in `uiState.utilities`, marks that something has run (which makes the Result subtab appear), and runs it.

State

```
uiState.utilities = {
  activeUtilityTabId:    "tab-tools" | "tab-lab" | "tab-result",
```

```
  lastUtilitySubtab:     "Tools" | "Lab",
  activeUtilityCategory: category name,
  activeUtilityItem:     the manifest entry,
  lastResult:            last Tools output (HTML),
  saved:                 null
}
```

Result is reproducible because the three remembered values rebuild the script's path.

Activation

Nothing calls `saveUtilityState()`, so `uiState.utilities.saved` stays null and `setUI.js` always calls `initUtilityTab(false)`. Init reloads the manifests, rebuilds the subtabs, and switches to `activeUtilityTabId`, which is remembered in `uiState`.

> **Probable glitch (read from the code, not tried).** Init with `restored = false` also resets "has anything run" to false, which hides the Result subtab. If you leave Utilities while on Result and come back, init switches to Result (re-running the script) while the Result subtab button is hidden.

Commands panel

Rebuild & Validate goes through the Node service, clears the manifest cache and marks every tab `needsUpdate`, then re-runs `initUtilityTab(false)` and prints the report.

After a manifest edit (Edit Manifest, Archive) the tab instead calls `refreshUtilitiesFromManifestEdit()`: it drops the cached Tools, Lab and Home manifests and reloads the two groups, finds the remembered item again by its path, and fails loudly if it is gone.

setUI.js — Switching Tabs

← Contents

> Rewritten 27 October 2026 against the current code. Changes since the earlier page: a third signal, `needsUpdate`, now comes first; `restore` clears the regions too (the old page said it did not); start-up also sets up the canvas layers; errors name the stage that failed. And one thing the old page implied but the code never does: `setUI` does not call any tab's `save()`.

`setUI.js` is the switchboard for the six main tabs. It renders nothing itself. It decides how a tab should be brought up, then hands over to that tab's *TabSpec*.

The routing table

```
TabRegistry = { home, draw, patterns, gallery, utilities, figures }
```

Each value is a TabSpec with at least `init(restored)` and `restore()`.

setUI(tab): what happens on a tab click

1. Turn off any Prev/Next overlay.
2. Reject an unknown tab name.
3. Write the tab name to session storage (`sketchpad.lastActiveTab`; nothing reads it back at present).
4. Apply the tab's theme class to `#wrapper`, before anything is built, so new DOM is styled right away.
5. `activateTab(tab)`: init or restore (below).
6. If a launch intent is pending (`uiState.launch.pending`), clear the pending flag first, then call `setUI(targetTab)` unless it is already active. Clearing first prevents a loop.

Each step records its name; if anything throws, the console says which step it was.

activateTab: init or restore

```
if  uiState[tab].needsUpdate      -> clear it, clearDivs(), init(true)
elif uiState[tab].saved            -> clearDivs(), restore()
else                               -> clearDivs(), init(false)
then uiState.activeTab = tab, highlight its button
```

- **needsUpdate** is set on every tab by `syncSystemStateAfterRebuild()` after a Rebuild & Validate, so each tab reloads its manifests the next time it is shown.
- **saved** is whatever the tab itself has written there. `setUI` never calls `save()`.
- Both paths start with `clearDivs()` (`uiUtilities.js`).

Which tabs actually restore

Tab	Who sets `saved`	Result
Home	pre-filled in `uiState.js`	always `restore()`
Patterns	the tab, from its first init on	`restore()` after the first visit
Gallery	the tab, from its first init on	`restore()` after the first visit
Draw	nobody	always `init(false)`, which itself returns to the open subtab
Utilities	nobody (`saveUtilityState` is never called)	always `init(false)`
Figures	nobody	always `init(false)`, which returns to the open figure

Either way, every tab comes back to what it showed, because each keeps its state in `uiState` as you work. The real difference is only which function rebuilds the screen. The end-of-work list in `PHASES.md` (items 4 and 6) proposes settling on one convention.

Start-up

On `DOMContentLoaded`: set up the menu manager, the panel overlays and the canvas layers; wire each main tab button to `setUI`; then `setUI("START_APP")`, which brings up Home (with `init`). If start-up fails, the message goes into `#text` so the page is not just blank. A page reload always starts fresh: nothing is read back from session storage.

The launch intent

One tab can ask for another to open with context, without calling it directly. Home does this for built-in Draw objects: it fills `uiState.launch` (pending, source, target, type, registry key) and calls `setUI("draw")`. Draw's init reads the intent, clears it, and opens the object.

uiState — The App's Memory

← Contents

> Rewritten 27 October 2026 against `ui/uiState.js`. The earlier page described a `uiState.manifests` area (manifests now live in `manifest.js`'s own cache) and a simpler restore rule; it did not know about `needsUpdate` or the Figures state.

`uiState` is one plain object that remembers where the user is: which tab, what each tab is showing, what is selected. It is declared in `ui/uiState.js`, exported, and also put on `window`, so code can use the bare name `uiState` anywhere.

Rules

- **Data only.** No functions, no DOM nodes. (A few tabs break this inside their own areas: Draw stores the registry entry itself in a subtab, and Figures keeps live drawing instances beside each figure.)
- **Declared, never invented.** Every value is declared in `uiState.js` with its default (CLAUDE.md: "a global value must be declared in its state object").
- **Written through immediately.** Tabs write state as the user works, so switching tabs needs no save step.
- **Not kept across a page reload.** A reload starts from the defaults.

What is in it

Part	Holds
`launch`	a one-shot request for another tab to open with context (Home to Draw). Cleared as soon as it is used.
`canvasWidth`, `canvasHeight`	800 by 800
`activeTab`	the main tab showing
`home`	`saved`: view, status, index, entry (pre-filled, so Home always restores)
`draw`	`activeSubtab`, `tabs` (one per open object), `saved: null`
`patterns`	`activeCategory`, `activeItem`, `saved`
`gallery`	`activeDomain`, `activeCategory`, `activeItem`, `activeSubtab`, `defaultSaved`, `saved`
`figures`	`activeSubtab`, `tabs` (one per open figure, each holding the figure and its instances), `hitRadius: 5`, `saved: null`
`utilities`	the last run: subtab, category, item, result; `saved: null`
`overlay`	whether an overlay is up, its type and title

Every tab area also has `needsUpdate`.

The three signals setUI reads

- **needsUpdate**: true after a Rebuild & Validate; the tab is re-initialized (with `restored = true`) and the flag cleared.
- **saved**: if set, the tab's `restore()` rebuilds the screen from it; if null, its `init(false)` does.
- **launch.pending**: after the tab is up, jump to `launch.targetTab`.

"Switching Tabs" (`setUI.html`) has the table of which tabs restore and which re-init.

Active versus saved

Within a tab's area, the `active...` fields say what is selected right now; `saved` is the recipe for rebuilding the screen on return. For Patterns and Gallery the tab writes both on every move. For Draw, Utilities and Figures, `saved` stays null and init rebuilds from the active fields and `tabs`.

Other window globals

The end of `uiState.js` lists the other values code reaches through `window`: `window.overlayManager`, and the third-party `window.Coloris` and `window.jQuery`. The interactor also puts `armInteractor` and `disarmInteractor` there, and the registry objects live in `window.drawRegistry`.

uiUtilities.js — Shared Screen Helpers

← Contents

> Rewritten 27 October 2026. The file is now `ui/uiUtilities.js` (it was `ui_utilities.js`; this page keeps its old name so existing links still work). The three overlapping offcanvas functions the earlier page complained about are now one, with two old names kept as aliases; the help-manifest loader has moved to `help.js`; canvas layers are cleared through `canvasLayerManager.js`.

A collection of helpers every tab uses: clearing the screen between tabs, the Commands button and its slide-in panel, thumbnail grids, and the two clean-ups that follow a Rebuild & Validate.

clearDivs(extraId)

The one teardown every tab switch goes through (`setUI.js` calls it before every `init` and `restore`). In order:

1. Empties `#action`, `#caption`, `#text`, `#sketchpad` (and `extraId`, if given) and resets their display.
2. Empties and hides the Figures layer strip (`#figure-sidebar`).
3. Removes everything in the permanent `#button` div tagged `tab-extra`: the per-tab additions such as Create Figure and Points Mode. (CLAUDE.md: anything a tab adds to `#button` must carry this class.)
4. Empties `#interface-controls`.
5. Hides `#sketchpad-wrapper`: a tab that draws must show it again.
6. Clears every canvas overlay layer (`canvasLayerManager.clearAll()`) and the main canvas.
7. Disarms the interactor.

After a Rebuild & Validate

- **syncSystemStateAfterRebuild()**: empties the whole manifest cache and sets `needsUpdate` on every tab, so each re-initializes the next time it is shown. This is the one the Commands panels use.
- **rebuildAllSystemCaches()**: empties the cache, reloads every known manifest at once, resets Home's grouping, and deletes every tab's `saved`. Nothing calls it at present.

The Commands button and its panel

- `setCommandsButtonLabel(label)`: the button's text ("Figures Commands").
- `setCommandsButtonHandler(onClick)`: what it does; replaces any earlier handler. Every tab uses this one.
- `setCommandsButton(label, onClick)`: an older way that replaces the button element itself; not used.
- `showOffcanvas({ title, buildBody | bodyHtml })`: shows the permanent slide-in panel (`#offcanvasPanel`, Bootstrap) with a title and either a function that fills the body (Commands

panels, which then wire their buttons) or ready HTML. `showCommandsOffcanvas` and `showOffcanvasPanel` are the old names, kept as aliases.

- `formatRebuildReportShared(report)`: turns the Node service's rebuild report into the text the panels show.

Thumbnails

- `renderThumbnailGrid(targetId, items, buildSrc, onClick)`: a grid of `img.thumb-image` in a `.thumb-panel` (Patterns, Gallery).
- `markSelectedThumbnail(targetId, index)`: puts `thumb-selected` on one of them.
- `buildCanvasThumbnailBase64(canvas, w, h)`: crops a canvas to its drawn area and returns a small PNG, for Create Thumbnail and secondaries.

Passed through from elsewhere

- `buildCategoryDescriptor`, from `categories.js`.
- `syncOverlayToCanvas(name, canvas)`, which is `syncLayerToCanvas` from `canvasLayerManager.js`: sizes an overlay canvas to match the drawing canvas.
- `escapeHtml(text)`: makes text safe to put into HTML.

Not used

`clearOverlaysForTabSwitch()` (hides the overlay container and clears the panels), `setCommandsButton()` and `rebuildAllSystemCaches()` are exported but nothing calls them.

The Manifest System: Loading, Caching, Invalidation, and Tab Registration

← Contents

> Checked 27 October 2026 against the current code; this page was already close. Corrected: paths are absolute (`/<basedir>/...`, no longer `../`); there is a sixth public method, `getRegistry()`; Figures registers a callback but reads its category list with `fetch()` itself and has no archive or edit command yet, so nothing invalidates `"figures"`; the Node service file is `node/sketchpadService.mjs`.

This treatise describes the current manifest architecture as implemented across `ui/manifest.js`, the tab lifecycle files, and `ui/home/homeManifest.js`. It supersedes the earlier treatise, which documented a simpler system without the registration/invalidation model.

> **Scope**
> 1) The `ManifestManager` class: its four caches and six public methods.
> 2) How tabs register callbacks and how invalidation fires them.
> 3) How the standard tabs (Patterns, Gallery, Utilities, Figures) use the system.
> 4) How the Home tab differs and why.
> 5) The two global-reset operations that bypass the registration model.
> 6) How the Node service keeps `/home/manifest.json` consistent when items in other tabs are edited or archived.

1. ManifestManager: the four caches

`ManifestManager` is instantiated once as a singleton named `manifest` and imported by every file that needs manifest data. Its constructor initialises four dictionaries:

```
this.cache        = {};  // basedir → raw loaded data (array or array-of-arrays)
this.registryCache = {}; // basedir → directoryRegistry array
this.shapedCache   = {}; // basedir → { categoryName: [entries] }  (lazily built)
this.listeners     = {}; // basedir → [ callback, ... ]
```

All four are keyed by **basedir** — the string passed to `manifest.get(basedir)`. Examples of valid basedir strings:

```
"patterns"
"gallery/Ideabook"
"gallery/Patterns"
"gallery/Scripts"
"utilities/Tools"
"utilities/Lab"
"figures"
```

Home is a special case and is discussed separately in section 5.

2. The six public methods

2.1 get(basedir)

The primary loader. Checks `cache` first; on a miss it loads from disk via `fileLayer` using one of two models:

- **DirectoryRegistry model** — if `/<basedir>/directoryRegistry.json` exists, it is read as an array of category names. For each category, `/<basedir>/<category>/manifest.json` is loaded (or an empty array substituted if absent). The result is an array-of-arrays, one inner array per category. Both the result and the registry array are cached.
- **Flat model** — if no registry exists but `/<basedir>/manifest.json` exists, it is loaded as a flat array and cached.
- **Nothing** — returns and caches `[]`.

All tabs call `await manifest.get(basedir)` during their init to prime the cache before calling `getCategoryMap`.

2.2 getCategoryMap(basedir)

Synchronous. Returns a plain object `{ categoryName: [entries], ... }` shaped from the raw array-of-arrays. The shaped result is memoised in `shapedCache` and cleared automatically by both `invalidate` and `clearCache`. Throws if `get(basedir)` has not yet been awaited.

This is the method tabs use when they need to look up items by category name. It replaces the earlier pattern of tabs maintaining their own `manifest.cache.patterns` secondary cache.

2.3 register(key, callback)

Registers a zero-argument callback to be fired whenever `invalidate(key)` is called for that key. Multiple callbacks can be registered for the same key. Each tab registers once, guarded against duplicate registration on repeated inits:

```
if (!initPatternsTab._registered) {
  manifest.register("patterns", () => restorePatternsTab());
  initPatternsTab._registered = true;
}
```

Registrations accumulate in `listeners[key]` and are never automatically cleared — they persist for the lifetime of the page. The duplicate guard on the `_registered` flag ensures each tab registers exactly once regardless of how many times its init function is called.

2.4 invalidate(key)

The primary mechanism for signalling that on-disk data has changed for a specific key. It does three things in order:

1. Deletes `cache[key]`, `registryCache[key]`, and `shapedCache[key]`.
2. Logs the invalidation to the console.
3. Fires every callback registered for that key.

The callbacks fire synchronously, so they execute before `invalidate` returns to its caller. Async callbacks (those that return a Promise) are not awaited — they run in parallel with whatever

follows the `invalidate` call. This is intentional: the tab doing the edit does its own navigation independently, and background tabs refresh themselves without blocking the foreground work.

2.5 clearCache()

A global reset: wipes all three data caches (`cache`, `registryCache`, `shapedCache`) in one operation. It does **not** touch `listeners` — registrations survive a global reset. It does **not** fire any callbacks.

`clearCache` is reserved for two specific global-reset operations described in section 6. All other invalidation should use `invalidate(key)`.

3. How the standard tabs use the system

Patterns, Gallery and Utilities all follow the same pattern. Figures registers a callback (below) but does not use `manifest.get()`: `figuresDisplay.js` fetches `/figures/directoryRegistry.json` and each category's manifest itself. `getRegistry(basedir)` returns the cached category list for a basedir (an empty array if none).

3.1 At init time

1. Call `await manifest.get(basedir)` to prime the cache.
2. Call `manifest.getCategoryMap(basedir)` to get the shaped data.
3. Register a refresh callback (once, guarded by `_registered`).

3.2 After a Node operation that mutates a manifest

The tab's menu command calls `manifest.invalidate(key)`. This clears the stale cache and fires the registered callback, which reloads and re-renders. The calling code then handles its own navigation (e.g. deciding whether to stay in pattern view or return to categories after an archive).

3.3 Registration table

Tab	Key(s) registered	Callback
Patterns	`"patterns"`	`restorePatternsTab()`

Gallery	`"gallery/Ideabook"`
	`"gallery/Patterns"`
	`"gallery/Scripts"`
		`restoreGalleryTab()` (all three)

Utilities	`"utilities/Tools"` `"utilities/Lab"`	`refreshUtilitiesFromManifestEdit()` (both)
Figures	`"figures"`	`initFiguresTab(false)`
Home	`"home"`	`loadHomeManifest_async()` — only if Home is the active tab (`uiState.activeTab === "home"`)

3.4 Invalidation table: which keys are invalidated and when

Operation	Keys invalidated
Edit manifest entry (Patterns)	`"patterns"`, `"home"`
Archive item (Patterns)	`"patterns"`, `"home"`
Edit manifest entry (Gallery)	`"gallery/Ideabook"`, `"gallery/Patterns"`, `"gallery/Scripts"`, `"home"`

Operation	Keys invalidated
Edit manifest entry (Utilities)	`"utilities/Tools"`, `"utilities/Lab"`, `"home"`
Archive item (Utilities)	same as above (routes through `refreshUtilitiesFromManifestEdit`)
Archive or edit manifest (Figures)	none yet: Figures has no such commands until Save As (Phase 10)
Create pattern PNG (from Draw)	`"gallery/Patterns"`
Create pattern script (from Draw)	`"patterns"`
Edit manifest entry (from Home)	`"home"` (via `refreshHomeCategoriesFromManifestEdit`)

4. How Home differs

Home's manifest is architecturally distinct from all other tabs. The differences are not incidental — they follow from what Home actually represents.

4.1 What Home's manifest is

`/home/manifest.json` is a **virtual aggregate**: a flat array assembled by the Node rebuild service by scanning all status-bearing entries across every Patterns, Gallery, and Utilities manifest on disk. It is not the authoritative source for any item — the per-tab manifests are. Home's manifest is a derived view, rebuilt from scratch whenever `rebuildAndValidateManifests` runs.

4.2 Why Home bypasses manifest.get()

`manifest.get("home")` would first probe for `/home/directoryRegistry.json`, which does not exist. The dev server returns a 200 with an HTML error page rather than a 404 for unknown paths, causing `fileLayer.exists()` to return true and `fileLayer.loadJSON()` to fail on parsing.

Home's loader (`loadHomeManifest_async` in `homeManifest.js`) therefore bypasses `manifest.get()` entirely and calls `fileLayer.loadJSON("/home/manifest.json")` directly. It then stores the result manually into `manifest.cache["home"]` so that `manifest.invalidate("home")` can clear it in the normal way.

```
// loadHomeManifest_async — simplified
if (Object.prototype.hasOwnProperty.call(manifest.cache, "home")) {
  data = manifest.cache["home"];          // cache hit
} else {
  data = await fileLayer.loadJSON("/home/manifest.json");  // disk read
  manifest.cache["home"] = data;          // store manually
}
```

4.3 Home's registration callback

Home registers a callback for `"home"` that only runs when Home is currently the active tab:

```
manifest.register("home", () => {
  if (uiState.activeTab === "home") {
    loadHomeManifest_async();
  }
});
```

When another tab (e.g. Gallery) calls `manifest.invalidate("home")`, the cache is always cleared. If Home is active, the callback fires `loadHomeManifest_async()` immediately so the user sees the updated data without navigating away. If Home is in the background, the cache is already empty and the next time the user switches to Home it will load fresh data from disk.

4.4 Home's data shape

Unlike other tabs, Home does not use `getCategoryMap`. After loading, `loadHomeManifest_async` passes the flat array to `groupHomeEntriesByStatus(data)`, which produces a plain object keyed by status string. Only entries that have a non-empty `status` field appear in this object — entries without a status exist in the flat array on disk but are invisible to Home's UI.

```
{
  "working":  [ { file, path, title, status }, ... ],
  "favorite": [ ... ],
  "broken":   [ ... ]
}
```

This grouped object is stored in `homeState` via `setHomeManifestGrouped()` and drives Home's category frames.

5. Keeping Home consistent after per-tab edits

Because `/home/manifest.json` is a derived file, any edit to a per-tab manifest that adds, changes, or removes a status must also be mirrored into Home's manifest on disk. This is handled entirely in the Node service (`node/sketchpadService.mjs`) so the browser does not need to know about it.

5.1 editManifestEntry (Case A: editing from within a tab)

When the service writes a status change to a Patterns, Gallery, or Utilities manifest, it immediately mirrors that change into `/home/manifest.json` handling three cases:

- **Status newly set** — the item does not yet exist in Home's manifest. A new entry is appended.
- **Status changed** — the item already exists in Home's manifest. Its `status` and `title` are updated in place.
- **Status cleared** — the item exists in Home's manifest. It is removed.

The match is made by **rooted path** — the item's absolute path from the project root, e.g. `/gallery/Scripts/Arcs/arcsAndInner.js`. This is computed by the `makeRootedPath` helper from the item's manifest directory and its local `entry.path` value. The response includes a `homeMirrorResult` field (`"added"`, `"updated"`, `"removed"`, or `"skipped"`) for diagnostics.

5.2 editManifestEntry (Case B: editing from within Home)

When the UI sends a manifest edit with `manifestPath` pointing to `/home/manifest.json`, the service treats this as a virtual edit. It uses the rooted path (the `matchValue`) to locate the original per-tab manifest entry, updates that file, and then also updates `/home/manifest.json` directly.

5.3 archiveItem

When an item is archived from any tab, the service removes it from the item's own manifest and also removes the matching entry from `/home/manifest.json`, again matched by rooted path. Both

writes happen before the service returns, so the browser's subsequent `manifest.invalidate("home")` always finds correct data on disk.

6. Global reset operations

Two operations wipe the entire cache rather than invalidating specific keys. Both are in `uiUtilities.js` and are only called after the Node rebuild service has rewritten all manifests on disk.

- **`syncSystemStateAfterRebuild()`** — clears the cache and marks all tab `needsUpdate` flags so each tab re-initialises on next visit.
- **`rebuildAllSystemCaches()`** — clears the cache then immediately re-fetches all known basedirs in parallel to warm the cache proactively.

Neither fires registered callbacks — that is intentional. A global rebuild replaces all data from scratch and the tab lifecycle (via `needsUpdate`) handles re-rendering. Firing all callbacks would cause redundant re-renders before the fresh data arrives.

`clearCache()` should not be called outside of these two functions. All other invalidation uses `invalidate(key)`.

7. End-to-end example: editing a Gallery/Scripts status

1. User opens a Gallery/Scripts item, chooses Edit Manifest, sets status to "working", confirms.
2. UI calls `openEditManifestDialog()` which dispatches `editManifestEntry` to the Node service.
3. Node writes the new status to the item's manifest file, then mirrors it into `/home/manifest.json` (adding a new entry because none existed). Returns `homeMirrorResult: "added"`.
4. UI calls `refreshGalleryFromManifestEdit()`, which calls:
   - `manifest.invalidate("gallery/Ideabook")`
   - `manifest.invalidate("gallery/Patterns")`
   - `manifest.invalidate("gallery/Scripts")` — fires `restoreGalleryTab()`
   - `manifest.invalidate("home")` — fires Home's callback
5. Home's callback checks `uiState.activeTab`:
   - If Gallery is active: cache is cleared, Home loads fresh on next visit.
   - If Home is active: `loadHomeManifest_async()` runs immediately, reads the updated file, re-groups by status, and re-renders.

8. Summary: what each layer owns

Layer	Responsibility
`manifest.js`	Loading, caching, shaped access, registration, and targeted invalidation. Knows nothing about tabs or UI.
Tab init functions	Call `manifest.get()` to prime the cache, then register a refresh callback once.

Layer	Responsibility
Tab menu command functions	Call `manifest.invalidate(key)` after Node operations, including `"home"` for any status-affecting operation. Handle their own navigation independently of the callback.
`homeManifest.js`	Loads Home's flat manifest directly via `fileLayer`, stores into `manifest.cache["home"]` manually, groups by status, and updates Home's display state.
`node/sketchpadService.mjs`	Writes per-tab manifests and mirrors every status change into `/home/manifest.json` immediately, so the browser never reads stale home data after a cache clear.
`uiUtilities.js`	Owns the two global-reset operations (`clearCache()`) that follow a full Node rebuild. These do not fire callbacks.

End of treatise.

fileLayer.js — Reading Files and Building Paths

← Contents

> Rewritten 27 October 2026 against the current code. The main changes since the earlier page: every path is now absolute (`/patterns/...`; the old `../patterns/...` forms are gone and a relative path is refused), `exists()` uses GET instead of HEAD, and the path helpers have been adopted by `manifest.js`.

`fileLayer` is the browser's read-only way to reach files through the local web server, plus one place where paths are built so every tab builds them the same way. It knows nothing about tabs, the screen, or what the files mean. (Writing files is the Node service's job; see `nodeLayer.js`.)

Reading

Function	Returns	On failure
`exists(path)`	true if a GET succeeds	false (and a console error); a relative path is an error, also reported as false
`loadText(path)`	the text	`null`
`loadJSON(path)`	the parsed JSON	`null` (bad JSON included)
`listDirectory(path)`	the `href` values of a server directory listing	`[]`
`helpExists(tab, item)`	`exists("/help/<tab>/<item>.html")`	false

> **A trap with exists().** The Vite development server answers an unknown path with `index.html` and status 200, so `exists()` can say true for a file that is not there. That is why the Home tab reads its manifest directly instead of letting the manifest system probe for a directory registry that does not exist, and why `scriptRunner.js` checks the content type before importing a script.

`listDirectory` depends on the server producing an HTML listing. Its one caller is a `drawRegistry` branch in `manifest.get()` that nothing reaches (no code asks for `manifest.get("drawRegistry")`), and that branch would find nothing anyway: it passes a relative path and filters the returned strings for an `isDirectory` field they do not have.

Building paths: fileLayer.path

Helper	Gives
`directoryRegistry(basedir)`	`/<basedir>/directoryRegistry.json`
`categoryManifest(basedir, category)`	`/<basedir>/<category>/manifest.json`
`flatManifest(basedir)`	`/<basedir>/manifest.json`

`helpHtml(tab, item)`	`/help/<tab>/<item>.html`
`script(tab, item)`	`/<tab>/<item>.js`
`patternScript(category, file)`	`/patterns/<category>/<file>.js`
`galleryScript(file)`	`/gallery/Scripts/<file>.js`
`utilityScript(category, file)`	`/utilities/<category>/<file>.js`

The script helpers take a name with or without `.js` and always give exactly one.

In practice only the three manifest helpers are used (by `manifest.js`). The tabs still write their own script and help paths inline (for example `` `/patterns/${category}/${filename}.js` `` in `patternsDisplay.js`). Moving them onto these helpers is unfinished work, not a problem: the paths agree.

Old names

`normalizeHelpPath`, `normalizeScriptPath`, `normalizePatternPath`, `normalizeGalleryScript` and `normalizeUtilityPath` each just call the matching `path` helper. Nothing calls them; they can go when convenient.

nodeLayer.js (Browser → Node Dispatch Bridge)

← Contents

> Checked 27 October 2026; still accurate. Two additions: besides the three named wrappers, other code calls `nodeDispatch()` directly with these request names: `editManifestEntry`, `archiveSecondaryObject`, `saveSecondaryObject`, `readSecondaryObject`, `getSecondaryManifest`, `getPrimaryObjectsWithSecondaries`, `writeGalleryPatternPng`, `writePatternFromDrawRegistry`, `writePatternThumbnail`, `writeHelpFile` and `openInEditor`. The service itself is `node/sketchpadServer.mjs` (the HTTP server on port 5174) and `node/sketchpadService.mjs` (the tasks).

This page is intentionally “raw help HTML”: it is written as a single self-contained page so you can drop it into your help system as-is, and/or split it later into smaller pages. The discussion is based on one file:

- `ui/nodeLayer.js` (browser-side)

> **Scope of this treatise**
> 1) What `nodeLayer.js` does and does not do.
> 2) The single transport primitive: `nodeDispatch(requestName, payload)`.
> 3) The “thin convenience wrappers” (`nodeListLogFiles`, `nodeReadLogFile`, `nodeRebuildAndValidateManifests`).
> 4) Failure behavior and how UI code should use results.

1. What nodeLayer is in the Sketchpad architecture

Sketchpad runs UI code in the browser, but some operations are fundamentally “Node-side”: scanning folders, rewriting manifests, writing thumbnails, reading local log files, and so on. The browser cannot do those things directly.

`ui/nodeLayer.js` exists to provide a single, consistent bridge:

- The UI calls a function in `nodeLayer.js`.
- `nodeLayer` sends a structured request to the local Node service.
- The Node service runs the task and returns structured JSON.
- The UI renders the JSON report (usually into `#text` in Utilities, or into a tab report panel).

> **Strict boundary rule**
> `nodeLayer.js` does NOT implement tasks.
> It only transports requests and returns results.

2. The single transport primitive: nodeDispatch()

The heart of the file is one function:

```
export async function nodeDispatch(requestName, payload = {})
```

Think of `nodeDispatch` as your “RPC call” (remote procedure call) to the Node service. It takes:

- `requestName` — a string identifying which Node task to run
- `payload` — an object carrying any task parameters

It always sends the same JSON envelope:

```
{
  requestName: requestName,
  payload: payload
}
```

2.1 Endpoint and method

The service endpoint is hard-coded:

```
POST http://localhost:5174/dispatch
```

The request uses:

- HTTP method: `POST`
- Header: `Content-Type: application/json`
- Body: JSON stringified envelope

> **Why hard-code?**
> This reflects the intended “local tool” model: Sketchpad runs a known local Node service. If you later want configurability, this file is the single place to introduce it.

2.2 Fail-fast error behavior

`nodeDispatch` is explicitly fail-fast on HTTP failures:

- If `response.ok` is false, it reads the response text and throws an Error.
- The error message includes the HTTP status, status text, and response body.

```
throw new Error(
  `nodeDispatch: HTTP ${status} ${statusText}: ${text}`
);
```

> **Practical implication**
> If the Node service is not running, or the endpoint is wrong, or the service rejects the request, the UI will get a thrown error (a visible console error unless the caller catches and renders it).

If HTTP succeeds, it returns:

```
return await response.json();
```

> **Contract expectation**
> The Node service is expected to always return valid JSON on success.

3. Higher-level helpers (thin wrappers)

Below `nodeDispatch`, this file defines a small set of convenience functions:

- `nodeListLogFiles(limit = 10)`
- `nodeReadLogFile(name)`
- `nodeRebuildAndValidateManifests()`

Each wrapper has one job:

- Choose the correct `requestName`
- Build the correct payload shape
- Return the JSON report from the Node service

> **Key architectural principle**
> UI code should call these named helpers instead of sprinkling raw requestName strings everywhere. That keeps your system consistent and makes it easier to rename tasks later.

3.1 nodeListLogFiles(limit)

Calls:

```
nodeDispatch("listLogFiles", { limit })
```

Intended usage:

- The UI requests a list of available log files.
- The UI displays the list (for selection) and/or renders it in Utilities.

3.2 nodeReadLogFile(name)

Calls:

```
nodeDispatch("readLogFile", { name })
```

Intended usage:

- The UI requests the contents of a specific log file.
- The Node service reads the file and returns content as JSON (likely text content plus metadata).
- The UI renders it in a viewer area (usually `#text`).

3.3 nodeRebuildAndValidateManifests()

Calls:

```
nodeDispatch("manifestMaintenance", {})
```

Intended usage:

- The UI requests the Node service to scan folders and rebuild/validate manifests.
- The Node service returns a structured report (what changed, what was fixed, what failed).
- The calling tab invalidates its cached manifest data and reloads (because the browser cannot auto-detect on-disk changes).

> **Important lifecycle note**
> Calling this Node task is only half of the operation.
> The other half is UI-side cache invalidation + reload. `nodeLayer` does not do that; the tab/controller must.

4. What nodeLayer assumes about the Node service

`nodeLayer` makes very few assumptions, but they are strict:

- The service is reachable at `http://localhost:5174/dispatch`.
- The service accepts a JSON envelope with `requestName` and `payload`.
- On success, the service returns valid JSON.
- On failure, the service returns a meaningful response body (text) so `nodeDispatch` can include it in the thrown error.

> **Separation of responsibility**
> `nodeLayer.js` does not validate payload schemas. If you want payload validation, it belongs either: 1) in the calling UI code (client-side validation), and/or 2) in the Node service (authoritative validation).

5. How UI code is expected to use results

The usage model is:

1. Call a `nodeXXX()` function.
2. Receive a JSON report object.
3. Render it in the UI (often as formatted text).

Example pattern:

```
import { nodeRebuildAndValidateManifests } from "./nodeLayer.js";

const report = await nodeRebuildAndValidateManifests();
// render report into #text
// then clear/reload caches and refresh the active view
```

> **Why “structured JSON” matters**
> It keeps Node tasks composable and testable: tasks can return detailed data, and the UI can decide how to display it without scraping strings.

6. Summary checklist

- **nodeLayer.js** is browser-side transport only: it does not implement any Node tasks.
- **nodeDispatch** is the single primitive: POST JSON to the dispatch endpoint and return JSON.
- **Fail-fast**: non-OK HTTP responses throw with status + response body.
- **Wrappers** provide stable names for Node requests so UI code doesn’t scatter request strings.
- **Cache refresh is not automatic**: after maintenance tasks, the calling tab must invalidate and reload its cached manifest data.

End of treatise.

categories.js — Category Rendering System (Description)

← Contents

> Checked 27 October 2026. Accurate; additions are at the end of the page.

categories.js implements Sketchpad's universal category rendering system. It is a pure DOM builder that renders category frames from a single, tab-supplied descriptor format. It has **zero** knowledge of manifests, tabs, Patterns, Gallery, Draw, or any other subsystem.

The calling tab is responsible for all data decisions (grouping, titles, item order, handlers). This module only validates the descriptor structure (fail-fast) and renders what it is given.

Descriptor format (required)

renderCategories() expects a descriptor array of category-frame objects:

```
[
  {
    title: "Category Name",
    items: [
      {
        name: "Item Name",
        onClick: function,     // required
        hasSubitems: boolean   // optional
      },
      ...
    ]
  },
  ...
]
```

Each descriptor entry corresponds to one rendered "category frame" with a header and a list of clickable items.

---

renderCategories(targetId, descriptor)

This is the main entry point. It clears a target container and renders a complete category UI into it.

Arguments:

- `targetId` – DOM id of the container (for example, "text")
- `descriptor` – array of category frame descriptors (see format above)

Fail-fast behavior:

- Throws if `targetId` does not exist in the DOM

- Throws if `descriptor` is not an array
- Throws if any frame descriptor is not an object
- Throws if `frame.items` exists but is not an array
- Throws if any item lacks `name:string` or `onClick:function`

Rendering sequence (high-level):

1. Clears the target container.
2. Creates an outer wrapper: `<div id="categories">`.
3. Sorts category frames by `title` (case-insensitive).
4. For each category frame:
   - Builds the frame DOM (header + content).
   - Optionally sorts items by `name` (case-insensitive).
   - Builds each item DOM and wires its click handler.

Category item click behavior:

- Marks the clicked item as active via `setActiveItem(targetId, itemName)`.
- Invokes the caller-supplied `itemDesc.onClick()` action.

Sorting behavior:

- Category frames are always sorted by `frameDesc.title`.
- Items are sorted by default, unless the frame descriptor explicitly sets:
  `sortItems: false`

---

setActiveItem(targetId, itemKey)

This function updates visual state after a click. It finds all elements with class `.item` inside the target container and applies:

- `.active` to the clicked item
- removes `.active` from all other items

It is fail-fast: it throws if the target container cannot be found.

DOM-building helpers (private)

- **buildCategoryFrameElement(title, count)**
  Builds the outer frame structure:

  ```
  <div class="category-frame">
    <div class="category-header">Title (count)</div>
    <div class="category-content"></div>
  </div>
  ```

  The header text includes an item count suffix like `(12)` when `count` is numeric.

- **buildCategoryItemElement(name, hasSubitems)**
  Builds an item row:

```
<div class="item">
  <span>name</span>
  <!-- optional chevron icon -->
</div>
```

If `hasSubitems` is truthy, a Bootstrap icon chevron is appended: `<i class="bi bi-chevron-right"></i>`.

Utility functions

- **clearCategoryFrame(targetId)**
  Clears the target container by setting `innerHTML = ""`. This function does **not** throw if the container is missing.
- **buildCategoryFrame(targetId)**
  Clears the target container, creates the outer `#categories` wrapper, appends it, and returns the wrapper element. This is rarely used because `renderCategories()` normally performs this work.

  This function is fail-fast and throws if the target container is missing.

Overall, **categories.js** is the “universal renderer” for category-style UIs in Sketchpad. Tabs provide a descriptor that defines what to show and what happens on click. This module validates the structure and renders it consistently, deterministically, and fail-fast.

Added 27 October 2026

> Checked against the current code. Everything above still holds; these are the additions.

buildCategoryDescriptor(groups, itemLabelFn, onClickFn) builds the descriptor for the common case: `groups` is `{ category: [entries] }` (what `manifest.getCategoryMap()` returns); each entry's label comes from `itemLabelFn(entry)`; a click calls `onClickFn(category, list, entry, index)`. Patterns uses it. (Also re-exported from `uiUtilities.js`.)

A quirk in it: it sorts each list with `[...list].sort(itemLabelFn)`, passing the label function where `sort()` expects a comparison of two items. The result is that the list is left in its original order. It works, because the index handed to the click matches the original manifest order that Patterns looks up, and `renderCategories()` then sorts what is shown by name anyway. Changing it to a real alphabetical sort would break that index, so leave it, or fix both together.

secondaryAction: an item may carry `secondaryAction: function`; the item then shows a small separate button that runs it without selecting the item. Draw uses this for the ">" that lists an object's saved secondaries.

caption.js — The Caption Bar

← Contents

> Rewritten 27 October 2026 against the current code. New since the earlier page: a Reset button option and the Prev/Next click zones over the picture. Three helpers the old page described are still exported but no longer called, and one of them would fail if it were (see the end).

The caption bar is the gray strip above the picture. Every tab builds it the same way, through `setCaptionBar()`: a title on the left, buttons on the right.

setCaptionBar(config)

config	Meaning
`targetId`	the container, normally `"caption"` (required; throws if missing)
`title`	text on the left
`onPrev`, `onNext`	if given, Prev and Next buttons
`overlayTargetId`	with both of the above: also turns on click zones over that region (left quarter = Prev, right quarter = Next; `nextPrevOverlay.js`). Without it the zones are turned off.
`onPrimary`	if given, a Reset button (meant for secondary objects; no tab passes it at present)
`onMenu(button, event)`	what the "v" button does; without it "v" is drawn grayed and inert

The bar is rebuilt from nothing on every call. The "v" button is always there. The title goes in a `span.caption-title`; the buttons in a `div.caption-buttons`, in the order Prev, Next, Reset, v. Tabs that add their own items to the bar (Home's file path, Figures' Line and Parabola buttons and layer name) find it by those class names, after building it.

Who uses Prev/Next and the click zones

- Patterns: buttons, and click zones over the picture.
- Gallery: buttons, and click zones over the picture (scripts) or the image (in `#text`).

Exported but unused

- `normalizeCaptionEntry(raw)`: returns `{ title, filename, category }` from loosely shaped data.
- `rebuildCaption(config)`: empties the container and calls `setCaptionBar`, which empties it anyway.
- `buildMenuHandler(tabName, context)`: returns a handler that calls `menuManager.open(tabName, context)`. That is the wrong argument list for today's `open(items, anchor)`, so it would throw. Nothing calls it; it is a candidate for deletion.

menuManager.js — The Caption Menu

← Contents

> Rewritten 27 October 2026 against the current code. The Help-item builder the earlier page described has moved to `help.js` (menuManager still offers it under the same name), it now walks a nested help manifest, and a missing page gives a "Create Help" item instead of a grayed one. The unused imports the old page pointed out are gone. Tooltips are new.

`menuManager` shows the small drop-down that opens from a caption bar's "v" button. It shows whatever list it is given; each tab decides what is in its menu.

Using it

```
menuManager.open(items, anchorElement)

items: [ { label, onClick, disabled?, tooltip? }, ... ]
```

Tabs build their menu fresh every time "v" is pressed, so an item's enabled state is always current. Most items come from small makers in `menuCmds.js`: `makeHelpItem`, `makeNotesItem`, `makeShowScriptItem`, `makeEditManifestItem`, `makeArchiveItem`, `makeThumbnailItem`.

What open() does

1. Refuses to run before `initMenuManager()` (called once by `setUI.js` at start-up).
2. Checks the list: an array; every item has a string label; every enabled item has an `onClick`. A bad list throws rather than drawing a broken menu.
3. Closes any menu already open: there is only ever one.
4. Builds a `div.caption-menu` with one `div.caption-menu-item` per item; a disabled item gets the `disabled` class and no click; `tooltip` becomes the hover text. Clicking an enabled item closes the menu, then runs it.
5. Adds it to `#wrapper`, 4 px below the anchor.
6. Closes on a press outside it (other than on the anchor) or on Escape; both listeners are removed when it closes.

The object

`menuManager` = `{ open, close, clear, createMenuItem, buildHelpItem, isReady }`. `clear` is another name for `close`; `createMenuItem(label, onClick, disabled)` makes one item.

The Help item: buildHelpItem(tabName, helpKey, options)

Lives in `help.js`. It reads `/help/manifest.json` (once, then cached), which is a nested object: tab, then subfolders, then a list of page names. It walks down `[tabName, ...options.subdirs]` (or `options.path`) and looks for `helpKey` in the list it reaches.

- Found: a **Help** item that opens `/help/<tab>/<subdirs>/<helpKey>.html` in the help overlay.

- Not found: a **Create Help** item that opens an empty editor for that path.

`makeHelpItem(tab, key)` in `menuCmds.js` wraps this, and gives a grayed "Help" when there is no key at all (Figures, for instance, passes none yet).

The help manifest is written by `node/generateHelpManifest.mjs`, which scans `/help`.

overlay.js — Help, Notes and Panel Overlays

← Contents

> Rewritten 27 October 2026 against the current code. The canvas layers (interaction, bbox, nodes, guides) that the earlier page described here have moved to `canvasLayerManager.js`. This file now handles only the panel overlays, and it has gained the Notes popup and an Edit button on help pages.

A panel overlay is an HTML panel shown over the app: a help page, a form, a debug panel, a file's notes. They all appear in one floating container (`#overlayContainer`, with a title bar and a close button) and take turns using it.

overlayManager

A small object (also on `window.overlayManager` for older code) with a list of named panels:

Method	Does
`register(name, element)`	adds a panel under a name
`show(name, html)`	fills the panel and shows it. **Also disarms the interactor**, so handles cannot be dragged under an open overlay.
`hide(name)`	hides it, keeping its contents
`clearLayer(name)`	empties and hides one panel
`clearAll()`	empties and hides every panel (not the canvas layers)

Unknown names throw.

initOverlay()

Called once at start-up (`setUI.js`). Registers `#overlay-help`, `#overlay-forms` and `#overlay-debug` as "help", "forms" and "debug", and wires the close button: it removes any per-overlay sizing from the panel, hides the container, and clears every panel. Missing elements throw at start-up.

Help pages: showHelpOverlay(helpPath, titleText, options)

- Fetches the page; a failed fetch throws with the status.
- Keeps only the page's `<body>` and shows it in the "help" panel, under a toolbar.
- The overlay's title is the page's own `<title>`; failing that, `titleText`; failing that, the file name.
- The toolbar has an **Edit** button (unless `allowEdit: false`) that opens the page in the TinyMCE editor (`help.js`).
- With `createIfMissing: true` it skips the fetch and opens an empty editor, for writing a page that does not exist yet.

Notes: showNotesOverlay(titleText, notesText, actions)

The Notes item in a caption menu shows a source file's header comments here, in the same panel and style as help. `actions` become toolbar buttons (Show Script, Open in VS Code); a button's handler may return a message, which is shown in a status line. `closeNotesOverlay()` closes it.

Small helpers

`showHelp(text)` and `hideHelp()` show or hide a short text in the help panel. `openOverlay(html)` and `closeOverlay()` are older names for the same thing.

Related

- `canvasLayerManager.js`: the four overlay canvases stacked over the drawing (see "Point Dragging" and the interactor page).
- `help.js`: the help manifest, the help home page, the Help menu item, the editor.

interactor.js — Handles You Can Drag

← Contents

> Rewritten 27 October 2026 against the current code. Since the earlier page: the handle presets changed (radius 5, grab range 10, red), handles can be selected and nudged with the arrow keys, they follow a layer's transform, the Figures centroid can take a press first, every move goes through one function (`moveHandle`) with before and after hooks for welds, and only the left button grabs. For the whole picture of dragging in Sketchpad (handles, discs, layers, welds) see "Point Dragging".

`interactor.js` draws an object's handles (its `params.points`) on the "interaction" overlay canvas and lets you drag them. One object is armed at a time. Draw, Figures and interactive scripts all use it.

Presets

PointPickerPresets	Value	Meaning
`handleRadius`	5	drawn size of a handle, canvas pixels
`hitTolerance`	10	how close a press must be to grab one
`idleColor`	`#C91A09`	red: a handle at rest
`activeColor`	`#FE8A18`	orange: hovered, dragged or selected

The public functions

- **armInteractor(instance)**: disarms any previous object, then starts listening for this one. Does nothing if the object has no `params.points`. Also on `window`.
- **disarmInteractor()**: removes the listeners, clears and hides the interaction canvas. Also on `window`.
- **refreshInteractor()**: redraws the armed object's handles; used when something moves them without going through the interactor (the Figures centroid).

What the armed object may supply

Property	Used for
`params.points`	the handles themselves, in object space (required)
`update(params)`	called after each move so the object rebuilds its geometry
`redrawHandler()`	called after each move to redraw (Figures sets this); without it the Draw tab's active subtab is redrawn, or the object's own `draw()`
`getTransform()`	the layer's transform (Figures). Handles are shown and hit in canvas space and stored in object space
`yieldTo(x, y)`	true if the press belongs to something above the handles (the Figures centroid)
`beforeMove()`, `afterMove()`	called around every move (Figures welds)

The mouse and keys

- **Left press** within 10 px of a handle: that handle is dragged and also becomes the *selected* handle (orange, with a black outer ring). A press elsewhere clears the selection. If `yieldTo` says the press belongs to something else, the handles stay out of it. Other buttons are ignored.
- **Move** while dragging: the handle is put exactly under the pointer (there is no "still a click" margin here; the smallest movement moves it). While not dragging, hovering turns a handle orange.
- **Release**: the drag ends; the selection stays.
- **Arrow keys**: move the selected handle 1 px (10 with Shift), in canvas directions. Ignored while the keyboard is in a text box, slider or list.

Listeners are on `window`, not the canvas, so a drag that strays off the canvas keeps going. All four are registered with one `AbortController`; disarming is one `abort()`.

moveHandle(i, x, y): the one place a handle moves

1. `beforeMove()`, if supplied.
2. Canvas (x, y) to object space through `getTransform()`; write into `points[i]`.
3. `onUpdate`: the object's `update()`, then its redraw.
4. `afterMove()`, if supplied.

Dragging and the arrow keys both go through it, so anything that has to react to a handle (welds) hooks in one place.

Things worth knowing

- The object's handle array is changed in place; the object reads it back in its own `update()`. Objects that keep extra handles in that array (a Line's or Parabola's midpoint) decide in `update()` what a moved midpoint means (they slide the whole line).
- Showing any panel overlay (help, forms) disarms the interactor (`overlayManager.show`).

Point Dragging

NEW

← Contents

> Written 27 October 2026. Sketchpad has four things you can drag, handled by three different files. This page puts them side by side, says which one wins when they overlap, and follows a drag through to the redraw.

The four draggable things

What	Looks like	Where	Code	What a drag changes
Handle	small red dot (orange when hovered or selected)	Draw, Figures, interactive scripts	`interactor.js`	one entry of the object's `params.points`; the object reshapes
Free disc	filled circle in the layer's color	Figures, points mode	`figurePointer.js`	that disc's position only
Centroid	blue diamond (orange when selected)	Figures, lines mode, active layer	`figureCentroid.js`	the layer's transform: the whole layer moves
Rotation handle	white circle on a stem above the box	Figures, while the centroid is selected	`figureCentroid.js`	the layer's transform: the whole layer turns about its centroid

An **anchored disc** is not draggable. It sits at a fixed fraction along its object's infrastructure and moves when the object does.

Who gets the press

All three files listen to the mouse at once. In order of precedence:

1. The **centroid** and its rotation handle (lines mode only). The interactor asks `yieldTo(x, y)` first and stays out if the centroid wants the press. A Line's midpoint handle sits exactly under its centroid, so on a Line the centroid always wins there.
2. A **handle** of the active object, within 10 px.
3. In points mode, a **free disc** under the press (within the larger of the hit radius and the disc's 9 px).
4. Otherwise, in points mode, the press is a click (place or select a disc).

Only the left button drags. Right-click in points mode is the weld toggle and drags nothing. (The centroid's handler does not yet check the button; a right-press there starts a move. Known, listed in `PHASES.md`.)

Click or drag?

- **Handles**: any movement drags. The handle is placed exactly under the pointer.
- **Points mode**: a press on a handle that moves more than 4 px is the handle being dragged; a press that stays within 4 px is a click (so a disc can be placed on a handle). A press anywhere else is always a click where it went down, even if the hand slips, because dragging empty canvas means nothing.

Arrow keys

- With a **handle** selected: the handle moves 1 px, 10 with Shift (Draw and Figures).
- With the **centroid** selected: the whole layer moves 1 px, 10 with Shift.
- Keys are ignored while typing in a text box, slider or list.

Object space and canvas space

In Figures every layer has a transform (move and turn). Handles and free discs are stored in the layer's own coordinates ("object space") and shown, hit and dragged in canvas pixels. Every drag converts the pointer position back through the transform before storing it, so a sideways drag moves the handle sideways on screen even on a layer that has been turned. The math is in `draw/drawTransform.js`.

A handle drag, start to finish (Figures)

1. `interactor.js` grabs the handle on the press.
2. Each mouse move calls `moveHandle()`:
   a. `beforeMove()` snapshots where this layer's welded discs are.
   b. The pointer position, in object space, goes into `params.points[i]`.
   c. The object's `update()` rebuilds its geometry; `redrawFigure()` redraws every layer and moves every anchored disc to its fraction along its arm.
   d. `afterMove()` compares with the snapshot. Each welded disc that moved passes the same move to its partners on other layers: one handle, both ends of an arm, or the whole layer, depending on where the partner sits (`figureWelds.js`).
3. Release ends the drag.

A centroid move or turn goes the same way, through `setTransform()` in `figureCentroid.js`, with the same snapshot-and-follow around it.

What a midpoint handle does

A Line and a Parabola keep a midpoint in their handle list. The object's `update()` notices that the midpoint moved and slides the whole line instead of stretching it. So dragging a line's middle moves the line; dragging an end reshapes it.

Where to look

Question	File
Why didn't the handle move?	`interactor.js` (armed? `params.points`? `yieldTo`?)
Why did a disc appear instead?	`figurePointer.js` (the click rules above)

Why did the whole layer move?	`figureCentroid.js`, or a weld fallback in `figureWelds.js`
Why didn't the partner follow?	`figureWelds.js` (is there really a weld? `figure.welds`)

Parameter Controls — Sliders and Their Kin

← Contents

> Rewritten 27 October 2026 against the current code. `ui/parameterControls.js` is now a 27-line forwarder; the real code is split by job under `ui/controls/`. The point-picker widgets the earlier page described are switched off (handles are dragged by `interactor.js` instead), and there are new widgets: number, accordion and thumbnail grid. The schema flags are listed on the companion page "Control Schema Flags" (`parameterControls2.html`).

Given an object's control schema and its current values, this system builds the controls in the left action panel and, when one changes, updates the value and asks for a redraw. Draw, Figures and runnable scripts all use it.

Where the code is

File	Job
`ui/parameterControls.js`	forwards everything from `controlsCore.js`, so old imports keep working
`controls/controlsCore.js`	`buildParameterControls()`, the public entry point
`controls/builders/dataBuilder.js`	schema plus values into one uniform list, for Draw-style and script-style sources
`controls/builders/groupFilter.js`	hides controls whose group is not selected (see the grouping guide)
`controls/rendering/renderer.js`	the list into DOM; `buildSingleControl()` picks the widget
`controls/rendering/rebuild.js`	rebuilds the whole panel (after a group switch)
`controls/widgets/*.js`	one file per kind of control
`controls/shared/`	widget names, CSS class names, small helpers

buildParameterControls(sourceInfo, targetTabId, render)

Two kinds of source:

- **Draw-style** (Draw, Figures): `{ drawRegistry, parameters, redrawHandler, onParamChange }`. The schema is `drawRegistry.controls`.
- **Script-style**: the script's `scriptInfo`, `{ controls, parameters, redrawHandler }`.

Which one is chosen: a `targetTabId` starting `"tab-draw"` means Draw-style, `"tab-scripts"` means script-style; anything else (including the default `"tab-generic"`) is decided by the source's shape (`controls` present: script; `drawRegistry` present: Draw). Figures passes `"tab-draw"`. The result is a normalized list, `{ key, label, widget, min, max, step, options, value, default, ... }`, drawn at once unless `render` is false.

Drawing the panel

- Writes into `#drawControls` inside `#action` (created if missing), emptied first, so other things in `#action` survive. `#action` must exist.
- Controls marked as interface controls go to `#interface-controls` instead.
- No controls: "(no parameters)".
- Each entry goes through `buildSingleControl()`: a `div.ctrl-field` with a label and the widget. A hidden widget returns nothing and takes no room.

The widgets

widget	What it is
`range`	slider with min / value / max (and < > step buttons); or the header style with `rangeHeader: true`
`rangeHeader`	"Label: value" above a full-width slider
`checkbox`	true / false
`select`	drop-down from `options` (strings or `{ value, label }`); numeric strings come back as numbers
`radio`	one of several, from `options`
`color`, `colorPicker`	color chooser, stores `"#rrggbb"`
`number`	a number box, respecting min and max
`button`	runs an action (a function, or a name looked up in `info.actions`)
`accordion`	collapsible sections
`thumbnailGrid`	a grid of thumbnails to pick from
`staticText`	text only; can be shown only when a boolean parameter is on (`showKey`)
`hidden`	nothing drawn
`text`, or anything unknown	a plain input box

`pointPicker` and `pointPickerArray` are still named in the constants, but their cases in `buildSingleControl` are commented out, so such a key falls through to a plain input box. Two registry objects still declare one (`boxes`, `regularPursuit`, for a midpoint).

When a control changes

1. The new value goes into `info.parameters[key]` (for Draw and Figures, that is the object's own `params`, so the drawing data changes directly).
2. `info.onParamChange()`, if there is one (Draw marks the subtab with "*"; Figures passes an empty function).
3. `info.redrawHandler()`.
4. If the control asks for it (`rebuildControls`, or `showsGroup` on a select or radio), the whole panel is rebuilt.

parameterControls.js — Control Schema Flags & Rendering Rules

← Contents

> Checked 27 October 2026; the flags below still work as described. The code now lives in `ui/controls/` (see "Parameter Controls"), and `ui/parameterControls.js` just forwards to it. Not listed below and also available: `widget: "number"` (honors `min` and `max`), `widget: "thumbnailGrid"`, and the grouping flags `showsGroup` and `belongsToGroup` (see the grouping guide). The point-picker widgets are switched off. Environment detection (section 11) applies only when no tab id is given; a tab id starting `"tab-draw"` or `"tab-scripts"` decides it outright.

This document summarizes all control-schema flags and behavioral rules used by **parameterControls.js**. These flags determine how controls are interpreted, normalized, and rendered across the Draw and Scripts tabs of the Sketchpad UI.

---

1. Range Controls (`widget: "range"`)

1.1 Numeric Range Configuration

- `def.min`, `def.max`, `def.step`
- Applied by both `setRangeControl()` and `setRangeHeaderControl()`.

1.2 `def.noButtons === true`

- Suppresses the small step buttons (< and >) normally placed beside the slider.
- Used by both range styles.

1.3 `def.rangeHeader === true`

- Selects the alternate “header-style” range UI.
- Handled by `setRangeControlUnified()`:
  - `true` → `setRangeHeaderControl()`
  - `false` → `setRangeControl()`
- Makes `range` and `rangeHeader` interchangeable via schema flag.

1.4 `def.rebuildControls === true`

- After a control changes value, the Action panel is rebuilt.
- Triggers:
  - `buildDrawParameterData(info)`
  - `renderParameterControls(info, data, tabId)`
- Supported by:
  - `setRangeControl()`
  - `setRangeHeaderControl()`

- `setRadioControl()`

---

2. Hidden Controls (`widget: "hidden"`)

- Control is not rendered and consumes no space.
- Implemented via `setHiddenControl()`.

---

3. Static Text (`widget: "staticText"`)

3.1 Conditional Visibility

- `def.showKey = "<boolean-parameter-key>"`
- If the referenced parameter is not `true`, the control is skipped.
- Implemented in `buildSingleControl()`.

---

4. Button Controls (`widget: "button"`)

4.1 `def.fullRow === true`

- Button spans both grid columns.
- Otherwise it appears in the control column only.

4.2 `def.redraw === true`

- After the button action runs, `info.redrawHandler()` is invoked.

---

5. Accordion Controls (`widget: "accordion"`)

5.1 `def.startOpen === true`

- Accordion section begins expanded.

5.2 `def.redrawOnToggle === true`

- Calls `info.redrawHandler()` when the accordion is opened or closed.

5.3 Optional CSS Overrides

- `sec.listClass` and `sec.itemClass`
- Used when rendering accordion item lists.

---

6. Select & Radio Controls

6.1 `def.options`

- Array of enumerated options.
- Used by `setSelectControl()` and `setRadioControl()`.

---

7. `info.onParamChange` (Optional)

- If present, it is called after parameter updates.

- No longer required by `setRangeHeaderControl()`.

---

8. Interface Controls (`def.control`)

Interface Controls affect UI state but are **not saved** with drawing parameters. They are rendered into `#interface-controls` instead of the main Action panel.

8.1 How to Define

```
controls: {
  radius: {
    widget: "range",
    min: 10, max: 100,
    label: "Radius"
  },

  showGrid: {
    control: "checkbox",
    label: "Show Grid Overlay",
    default: false
  }
}
```

8.2 Behavior

- Rendered separately from drawing parameters.
- Automatically excluded from saved JSON.
- Accessed via `this.params.showGrid` like any other parameter.

---

9. Rendering Pipeline Overview

9.1 `buildParameterControls()`

- Normalizes schema into a control-data array.
- Auto-detects environment (Draw vs Scripts).
- Delegates to:
  - `buildDrawParameterData()`
  - `buildScriptParameterData()`

9.2 `renderParameterControls()`

- Writes into the active tab's `#action` area.
- Clears and rebuilds `#drawControls`.
- Routes interface controls to `#interface-controls`.
- Skips hidden controls (null fields).

9.3 `buildSingleControl()`

- Dispatches to the appropriate `setXControl()` function.
- Handles conditional suppression for `staticText`.

- Preserves button metadata (`action`, `redraw`).

---

10. Range Header UI Notes

The header-style range control uses a two-row layout:

- Row 1: label + live value
- Row 2: full-width slider

It requires:

- `info.parameters`
- `info.redrawHandler()`
- `tabId` and `key`

---

11. Notes on Environment Detection

`buildParameterControls()` distinguishes Draw vs Scripts by inspecting the shape of `sourceInfo`:

- **Scripts**: `sourceInfo.controls` exists.
- **Draw**: `sourceInfo.drawRegistry` exists.

This allows the same renderer to support multiple tab types without hard-coding.

End of documentation.

Control Grouping System - Implementation Guide

← Contents

> Checked 27 October 2026; the mechanism works as described. Corrections: the code is now in `ui/controls/` (the filter in `builders/groupFilter.js`, the rebuild in `rendering/rebuild.js`, the triggers in `widgets/selectWidget.js` and `widgets/radioWidget.js`); there is no `figuresRunner.js` (Figures builds its controls the same way Draw does, so grouping works there too); the example file is `gallery/Scripts/Experimental/concentricCircles.js`, not `concentricStitch_grouped.js`.

What Was Implemented

A **declarative grouping mechanism** for conditional control visibility in `parameterControls.js`.

No changes were needed in scriptRunner or the Draw and Figures tabs: the grouping logic is entirely inside the parameter controls (`ui/controls/`).

How It Works

1. Declare a Trigger Control

Mark a `select` or `radio` control with `showsGroup: true`:

```
controls: {
  boundaryType: {
    widget: "select",
    options: ["circle", "polygon", "ellipse", "modulated"],
    showsGroup: true  // <-- This control determines which group is visible
  }
}
```

2. Group Your Controls

Add `belongsToGroup` to controls that should only appear for specific trigger values:

```
controls: {
  // Always visible (no belongsToGroup)
  centerX: { widget: "range", min: 0, max: 700 },

  // Trigger control
  boundaryType: {
    widget: "select",
    options: ["circle", "polygon", "ellipse"],
    showsGroup: true
  },

  // Polygon group (only when boundaryType === "polygon")
  groupPolygon: {
    widget: "staticText",
    text: "--- Polygon ---",
```

```
    belongsToGroup: "polygon"
  },
  sides: {
    widget: "range",
    min: 3, max: 8,
    belongsToGroup: "polygon"
  },

  // Ellipse group (only when boundaryType === "ellipse")
  groupEllipse: {
    widget: "staticText",
    text: "--- Ellipse ---",
    belongsToGroup: "ellipse"
  },
  ellipseRatio: {
    widget: "range",
    belongsToGroup: "ellipse"
  }
}
```

3. That's It!

When the user changes `boundaryType` to "polygon":

- All controls with `belongsToGroup: "polygon"` become visible
- All other grouped controls are hidden
- Controls without `belongsToGroup` remain visible

Matching Logic

The group name in `belongsToGroup` must **exactly match** the value of the trigger control:

```
boundaryType: { options: ["circle", "polygon"] }  // trigger values

sides: { belongsToGroup: "polygon" }  // matches when boundaryType === "polygon"
```

Multiple Trigger Controls

You can have multiple trigger controls in the same schema. Each one independently controls its own groups:

```
controls: {
  shapeType: {
    widget: "select",
    options: ["circle", "square"],
    showsGroup: true
  },
  fillType: {
    widget: "select",
    options: ["solid", "gradient"],
    showsGroup: true
  },
```

```
  // Shape groups
  radius: { belongsToGroup: "circle" },
  size: { belongsToGroup: "square" },

  // Fill groups
  color: { belongsToGroup: "solid" },
  startColor: { belongsToGroup: "gradient" },
  endColor: { belongsToGroup: "gradient" }
}
```

State Preservation

Hidden controls keep their values! When you switch from "polygon" to "ellipse" and back:

- `sides: 6` remains `6` even while hidden
- Parameter values persist in `sourceInfo.parameters`
- Only visibility changes, not state

Backward Compatibility

Scripts without `showsGroup` or `belongsToGroup` continue working exactly as before. The grouping system is **opt-in**.

Works Everywhere

This system automatically works in:

- **Gallery Scripts** (scriptRunner.js)
- **Draw objects** (drawRunner.js)
- **Figures** (figureLayers.js, through the same Draw-style controls)

No special integration code needed in any runner.

Implementation Details

Core Functions Modified

1. `filterControlsByActiveGroups()` - New helper function that filters control lists
2. `buildDrawParameterData()` - Applies group filtering before returning
3. `buildScriptParameterData()` - Applies group filtering before returning
4. `setSelectControl()` - Checks for `showsGroup` and calls `rebuildControls()`
5. `setRadioControl()` - Checks for `showsGroup` and calls `rebuildControls()`
6. `rebuildControls()` - New function that triggers full control panel rebuild

Rebuild Flow

```
User changes trigger control (e.g., boundaryType)
  ↓
Change handler detects def.showsGroup === true
  ↓
Calls rebuildControls(sourceInfo, tabId)
  ↓
```

```
Calls buildParameterControls(sourceInfo, tabId, true)
  ↓
buildScriptParameterData() or buildDrawParameterData()
  ↓
filterControlsByActiveGroups() filters the control list
  ↓
renderParameterControls() rebuilds the DOM
  ↓
Control panel updates (hidden controls removed, new controls appear)
```

Example: See `gallery/Scripts/Experimental/concentricCircles.js` for a complete working implementation using this system.

scriptRunner.js (Universal Script Loader + Execution Pipeline)

← Contents

> Checked 27 October 2026 against the current code. Changes since this page was written: tabs now call one function, `runScriptByPath(path, mode, options)`, which stops the previously running script (if it exports `stop()`), disarms the interactor, loads the new one, and sends it to `executeScriptToCanvas` (mode `"canvas"`) or `executeScriptToText` (mode `"text"`). The canvas route now takes options (`canvasRegionId`, `enableControls`, `controlsRegionId`, `clearBackground`, `backgroundColor`), fills the background white, runs `runPattern()`, then arms the interactor if the script's `scriptInfo` is interactive with `params.points`, and builds its parameter controls. The legacy `initPattern` / `drawPattern` fallback described below is gone. One mismatch: the file's header says `runPattern(ctx)` receives the context when it declares a parameter, but the code always calls `runPattern()` with no argument.

This page is intentionally “raw help HTML”: it is written as a single self-contained page so you can drop it into your help system as-is, and/or split it later into smaller pages. The discussion is based on one file:

- `ui/scriptRunner.js`

> **Scope of this treatise**
> 1) What this module is responsible for (and what it refuses to do).
> 2) The script execution contract (which exports a runnable script must provide).
> 3) Why loadScriptModule() probe-fetches before dynamic import (the “Vite returns index.html” problem).
> 4) The two execution routes: canvas output vs text output.
> 5) Exactly what is reset when a script runs (and what is not).

1. What this file is in the Sketchpad architecture

Sketchpad has multiple places where a user can “run a script”: Patterns items, Gallery Scripts items, Utilities Lab, and Utilities Tools. These scripts come from different tabs and different directory trees, but the core runtime requirement is always the same:

> Load an ES module by path, verify it is really a JavaScript module, then execute it in a deterministic and tab-appropriate way.

`ui/scriptRunner.js` is the single shared execution pipeline. Tabs are responsible for selecting the script path and deciding where it belongs. This module is responsible for:

- Loading ES modules by rooted path
- Fail-fast validation (missing file, wrong MIME type, missing exports)
- Executing the script into either the shared canvas or the `#text` region

- Resetting drawing state when (and only when) a canvas execution occurs

> **What it deliberately does NOT do**
> It does not interpret “meaning” of scripts, does not manage uiState, and does not decide which scripts exist or where they come from. That selection belongs to each tab/controller.

2. The execution contract (what scripts must export)

This module enforces a simple contract:

- **Preferred modern export:** `runPattern()`
- **Legacy drawing export:** `drawPattern(params)` (optionally with `initPattern()`)
- **Text-only utilities:** `render()`

In the current implementation, `loadScriptModule()` requires `runPattern()` to exist (fail-fast). The execution functions also include fallback logic for legacy patterns, but module loading itself is strict.

> **Practical interpretation**
> Treat `runPattern()` as the universal “entry point”. If a script is runnable in Sketchpad, it should export `runPattern()`.

3. loadScriptModule(path): reliable module loading

`loadScriptModule(path)` accepts a rooted browser path such as:

```
"/patterns/curve_stitch/parabola.js"
```

It performs five responsibilities in strict sequence:

1. **Validate input** (must be a non-empty string).
2. **Normalize to rooted URL** (ensure it begins with “/” and remove accidental “//”).
3. **Cache-bust in dev** by appending a query string timestamp.
4. **Probe-fetch** the URL with `fetch()`:
   - Verify HTTP success (`res.ok`)
   - Verify JavaScript MIME type (content-type contains “javascript” or “ecmascript”)
5. **Dynamic import** the module using the same URL (`import(/* @vite-ignore */ spec)`).

3.1 Why probe-fetch exists (the Vite fallback problem)

Vite (and many dev servers) will return `index.html` when a path is wrong. That produces confusing dynamic-import errors because the browser thinks it is importing JavaScript but receives HTML (text/html) instead.

Probe-fetch converts that into a fail-fast error with context:

- It checks `content-type` before importing.
- If it is not JS, it reads a short HTML snippet and includes it in the error.

> **Bottom line**
> Probe-fetch is the only reliable browser-side “does this module path actually exist” check in the presence of dev-server HTML fallbacks.

3.2 Contract enforcement at load time

After import, `loadScriptModule()` requires:

```
typeof mod.runPattern === "function"
```

If not present, loading fails immediately. This ensures every runnable script has a single, predictable entry point for Sketchpad.

4. executeScriptToCanvas(mod, title): controlled canvas execution

`executeScriptToCanvas(mod, title)` runs a loaded module into the shared canvas. It enforces a deterministic canvas environment:

1. Locate `#sketchpad` (fail-fast if missing).
2. Clear it (`pad.innerHTML = ""`).
3. Reattach the shared canvas (`pad.appendChild(window.drawCanvas)`).
4. Reset drawing state once, deterministically (`resetCanvas()`).
5. Execute the script once (prefer `runPattern()`).

> **What “reset once” means**
> Sketchpad expects `runPattern()` to do as many drawing calls as it wants. This helper’s role is only to clear and reset the environment one time per script run, not to interfere with internal script sequencing.

4.1 Legacy fallback path

If a module does not use `runPattern()` but does provide legacy exports, this function supports:

- `initPattern()` to provide params (optional)
- `drawPattern(params)` to perform drawing

If neither `runPattern` nor `drawPattern` exists, it throws.

5. executeScriptToText(mod, title): controlled text/HTML execution

`executeScriptToText(mod, title)` is used for scripts that produce output intended for the `#text` region (Utilities → Tools).

It is explicitly non-canvas:

- Does not touch `#sketchpad`
- Does not attach a canvas
- Does not call `resetCanvas()`

It clears `#text`, then executes:

- Preferred: `runPattern()` and use its returned string as HTML if provided
- Alternate: `render()` and use its returned string as HTML if provided

If no HTML is returned, it writes a small status message to #text and (in the “no usable output” case) throws an error.

6. Reset dependency: drawState.js and resetCanvas()

Canvas execution depends on a concrete import:

```
import { resetCanvas } from "/draw/drawState.js";
```

This is intentionally a rooted “.js” import so that the browser never falls back to index.html. The reset step is what makes repeated executions deterministic: each run starts from a clean draw-state baseline.

7. How tabs should use this module (end-to-end pattern)

The expected tab/controller usage pattern is:

1. Build a rooted script path (based on the tab’s manifest selection).
2. `const mod = await loadScriptModule(path);`
3. Choose output route:
    - `executeScriptToCanvas(mod, title)` for Patterns/Gallery/Lab
    - `executeScriptToText(mod, title)` for Tools

> **Division of responsibilities**
> Tabs decide *which* script to run and *where* its output should go. scriptRunner decides *how* to load it safely and execute it deterministically.

8. Summary checklist

- **Single pipeline**: all runnable scripts are loaded and executed through this module.
- **Fail-fast loading**: probe-fetch validates HTTP and MIME type before import().
- **Contract enforcement**: modules must export `runPattern()`.
- **Two execution targets**: canvas (clears `#sketchpad`, reattaches canvas, calls `resetCanvas()`) vs text (writes HTML into `#text`, does not touch canvas).
- **Determinism**: reset happens once per canvas execution, not per draw call inside the script.

End of treatise.

actionsRegistry.js — Caption Menu Actions Registry (Description)

← Contents

> Checked 27 October 2026. `ui/actionRegistry.js` still exists and `main.js` still loads it, but nothing uses `MENU_ACTIONS` any more. Caption menus are now built from the small item makers in `menuCmds.js` (`makeHelpItem`, `makeShowScriptItem`, `makeNotesItem`, `makeEditManifestItem`, `makeArchiveItem`, `makeThumbnailItem`); see the menuManager page. The description below is of the file as it is, but the registry is a candidate for deletion. Its Help builder now resolves to `help.js`, and nothing offers "Duplicate" (Draw's menu has Copy Object instead).

actionsRegistry.js defines a small, declarative registry of caption-menu actions used throughout the Sketchpad UI. It centralizes how menu items are constructed so that individual tabs (Draw, Patterns, Gallery, etc.) do not hard-code menu behavior.

The file exports a single object, **MENU_ACTIONS**, whose keys are action names (e.g. `help`, `showScript`, `duplicate`). Each action provides a `builder(menuContext)` function.

A `builder` does not execute the action directly. Instead, it returns a fully formed menu-item descriptor that can be passed to `menuManager.open()`. This keeps menu rendering and menu behavior cleanly separated.

The `menuContext` object is supplied by the calling tab and contains only the data that specific action needs. Different actions expect different fields in `menuContext`.

In effect, this file acts as a dispatch table: tabs select which actions they want by name, and `actionsRegistry.js` defines how those actions are built and wired.

Defined actions

- **help**
  A shared action used across multiple tabs. The builder delegates to `menuManager.buildHelpItem()`, which is manifest-driven.

  Expected `menuContext` fields:

  - `tabName` – the active tab (for example, "draw")
  - `itemName` – the registry key or item identifier

  The returned menu item opens the appropriate help content for the current tab and item.

- **showScript**
  A shared action that displays the source code for the current item in an off-canvas panel.

  Expected `menuContext` fields:

  - `scriptPath` – path to the JavaScript file to display
  - `itemName` – used as the title in the off-canvas header

The builder returns a menu item labeled “Show Script”. When clicked, it calls `showScriptOffcanvas()`, which fetches the script, escapes it, and renders it for viewing.

- **duplicate**
  A Draw-specific action. It does not know how duplication works internally.

  Expected `menuContext` fields:

  - `duplicateHandler` – a function supplied by `draw.js`

  The builder returns a menu item labeled “Duplicate”. Clicking it simply invokes the provided handler. This keeps draw-specific logic out of the shared menu system.

Overall, **actionsRegistry.js** provides a declarative, extensible mechanism for defining caption-menu behavior. New actions can be added in one place, and tabs can opt into them by name without duplicating wiring logic.

ui_callbacks.js — Retired

← Contents

> Checked 27 October 2026: `ui_callbacks.js` no longer exists in the code. This page is kept so that links to it still lead somewhere; it records what the file was and what replaced it.

`ui_callbacks.js` was an early routing layer: one place where generic events (a tab activated, a category or item chosen, a parameter changed, Prev/Next, Save, Reset) were forwarded to the right tab by name. Its own header called it "no orchestrators", but it did orchestrate: tab activation called each tab's init directly.

What replaced it

The old callback	Now
`onTabActivated`	`setUI.js` and each tab's TabSpec (`init`, `restore`)
`onCategorySelected`, `onItemSelected`	each tab builds its category list with an `onClick` per item (`categories.js` renders it)
`onParamChanged`	the parameter controls call the tab's `redrawHandler` and `onParamChange` directly (`ui/controls/`)
`onPrev`, `onNext`	passed to the caption bar per tab (`caption.js`)
`onSave`, `onResetParameters`	caption menu items built by each tab
`onHelpRequested`, overlay open/close	`overlay.js` and `help.js`

The lesson the old page drew still holds: a second routing layer beside the tabs drifts out of step with them. The current design keeps routing inside each tab.

Sketchpad Architecture (Detailed)

← Contents

> Checked 27 October 2026. Most of this page still holds. Corrections, made in place below: setUI does **not** call `save()` before leaving a tab (tabs keep `uiState` current as they go; see "Switching Tabs"); setUI clears the regions before `restore()` as well as before `init()`; a third signal, `needsUpdate`, forces a re-init after a rebuild; the shared helpers file is `uiUtilities.js`. The page predates the Figures tab; see "Figures Tab — A First Discussion" under Tabs.

This document is the detailed architectural reference for Sketchpad. It is intended to support implementation, debugging, and long-term consistency.

This treatise assumes the core Sketchpad design rules:

- UI state drives the UI; uiState is updated first, then rendering happens.
- Each tab is an independent subsystem that owns its regions.
- Tab activation follows TabSpec init/restore/save rules.
- Manifests are loaded once, cached, and treated as immutable.
- Fail-fast everywhere; missing data is a broken state, not a “skip” case.

Mini-TOC

- 1. Tab Lifecycle (TabSpec + setUI)
- 2. uiState (Authoritative UI State)
- 3. Manifest Layer (manifest.js + cache rules)
- 4. Categories Rendering (categories.js descriptor contract)
- 5. Caption Bar (caption.js + navigation + menus)
- 6. Menus (menuManager + per-tab menuCmds shaping)
- 7. Script Execution (runPattern + scriptRunner)
- 8. drawRegistry Architecture (init/update/draw + elements)
- 9. File Identity + Paths (fileLayer.js)
- 10. Node Dispatch (nodeLayer.js)
- 11. Node Service (sketchpadService.mjs)
- 12. Manifest Maintenance (maintain_manifests.mjs)
- 13. Per-Tab Ownership Summary
- 14. Diagnostics and Failure Modes (debug map)
- 15. Cross-Functional Flows (end-to-end handoffs)

---

1. Tab Lifecycle (TabSpec + setUI)

Sketchpad is organized around top-level tabs. Each tab is a self-contained subsystem that is activated through a TabSpec object.

1.1 TabSpec contract

Every tab provides a TabSpec with these core lifecycle methods:

- `init(restoredFlag)` — cold-start initializer only
- `restore()` — rebuild from saved uiState only
- `save()` — snapshot current state before leaving tab

The TabSpec also declares which regions it owns (caption, text, sketchpad, action).

1.2 Cold start vs restore

Sketchpad distinguishes two entry cases:

- Cold start: the tab has no saved state yet, so it builds its UI from defaults.
- Restore: the tab has saved state, so it reconstructs exactly what the user was viewing.

Architectural rule: init() must not contain restore logic. restore() must not contain cold-start logic.

1.3 setUI.js responsibilities

setUI.js is the orchestrator. It must:

- switch active tab
- apply theme
- clear regions on cold init
- call init() or restore()
- (not done: setUI never calls `save()`; each tab writes its state into `uiState` as it goes)

Architectural rule: setUI.js does not render tab content. It only delegates to the active tab's controller logic.

Back to Mini-TOC

---

2. uiState (Authoritative UI State)

uiState is the authoritative record of what the user is looking at right now. It is the only "truth source" for restore behavior.

2.1 uiState as the driver

Sketchpad follows the rule:

```
User action → uiState update → rendering
```

Rendering never "discovers" state by reading the DOM. State is written first and then the DOM is rebuilt.

2.2 Saved state snapshots

Each tab maintains a saved snapshot:

```
uiState.<tab>.saved
```

This snapshot must contain enough information to reconstruct the exact view:

- view type (categories vs results vs object workspace)

- active category name
- active index or item identity
- active subtab if the tab has internal subtabs

Architectural rule: restore() uses only uiState.<tab>.saved. It must not rely on scattered flags or implicit UI state.

Back to Mini-TOC

---

3. Manifest Layer (manifest.js + cache rules)

Manifests define content inventory. They are the authoritative “what exists” layer for file-based tabs.

3.1 What manifest.js owns

manifest.js provides:

- loading JSON manifests by tab/domain name
- loading directory registries
- caching loaded results
- explicit cache invalidation

3.2 Cache rules

Architectural rules:

- Manifests are loaded once and cached.
- Cached manifest data is treated as immutable.
- Tabs only read cached data.
- Cache is invalidated only when a maintenance task changes inventory.

3.3 Why immutability matters

Treating manifest cache as immutable prevents subtle drift between:

- what the UI believes exists
- what the filesystem actually contains

When a maintenance task changes the filesystem, the correct response is:

1. invalidate cache
2. reload manifest
3. restore view from uiState

Back to Mini-TOC

---

4. Categories Rendering (categories.js descriptor contract)

categories.js is a pure renderer for category frames and items. It is intentionally ignorant of tabs, manifests, and filesystem structure.

4.1 Input descriptor format

Tabs pass an array of frames, where each frame contains:

- frame title
- items array

Each item includes:

- display name
- hasSubitems flag (optional)
- onClick handler (required)

4.2 Why this matters

This design keeps category rendering stable while tabs evolve independently.

Architectural rule: categories.js does not “decide behavior.” Tabs decide behavior by wiring onClick handlers.

Back to Mini-TOC

---

5. Caption Bar (caption.js + navigation + menus)

caption.js builds the caption bar for the active view. It is a shared UI component used across multiple tabs.

5.1 Responsibilities

caption.js is responsible for:

- displaying a title string
- wiring Prev / Next buttons when provided
- opening a menu when provided
- optionally enabling the Next/Prev click-zone overlay

5.2 Overlay target anchoring

Some views display content in #text (images). Some views display content in #sketchpad-wrapper (canvas host).

The caption bar must specify which region receives the overlay:

- overlayTargetId = "text"
- overlayTargetId = "sketchpad-wrapper"

Architectural rule: caption.js becomes the single on/off switch for the overlay.

5.3 Menu wiring

The caption menu is invoked by passing an onMenu callback:

- tab builds an info object describing current selection
- tab asks a per-tab menu builder for menu items
- menuManager opens the menu anchored to the caption control

Back to Mini-TOC

6. Menus (menuManager + per-tab menuCmds shaping)

Sketchpad menus are structured so that:

- tabs shape context
- shared commands perform work

6.1 menuManager.js

menuManager owns:

- menu rendering
- menu positioning
- closing behavior

6.2 Per-tab menu builders

Each tab provides a function like:

```
getPatternsCaptionMenuItems(info)
```

This function converts the current view context into menu items.

6.3 Shared menu commands

Shared menu command modules perform:

- nodeDispatch calls
- overlay open/close
- manifest cache invalidation
- UI refresh from uiState

Architectural rule: per-tab menu modules should be thin “shapers.”

Back to Mini-TOC

7. Script Execution (runPattern + scriptRunner)

Script execution is the bridge between:

- file-based content inventory
- drawRegistry-like runtime behavior

7.1 runPattern() as the canonical entry point

runPattern() is the orchestrator entry point for Sketchpad scripts.

Required runPattern sequence:

1. printTitle(scriptInfo.title)
2. alias setup: scriptInfo.parameters = scriptInfo.params
3. init() called once
4. buildParameterControls(scriptInfo, targetId, true)

5. explicit first draw (usually redrawHandler)

Architectural rule: runPattern() contains no drawing math.

7.2 scriptRunner.js responsibilities

scriptRunner handles:

- importing modules by path
- executing the correct entry point
- providing consistent error behavior

scriptRunner must not:

- mutate uiState
- interpret tab-specific meaning
- make manifest decisions

Back to Mini-TOC

---

8. drawRegistry Architecture (init/update/draw + elements)

drawRegistry entries are internal “first-class objects.” They are defined as predictable window globals and follow a uniform lifecycle.

8.1 The registry object shape

A typical entry includes:

- metadata fields (name, id, category, tags, description)
- visual fields (background, overlays, transforms)
- params (JSON-safe defaults)
- controls (UI metadata)
- elements (runtime objects)
- init/update/draw functions

8.2 elements as persistent runtime state

elements must be created once in init() and then mutated.

Canonical shape:

```
elements = { element: { ... } }
```

draw() reads only from elements.element. It does not read directly from params.

8.3 update(params) rules

update(params) is responsible for:

- copying values into elements.element
- type normalization (numbers, booleans, points)
- validation (fail-fast)
- recomputing derived values

update(params) must not:

- draw
- build UI
- create a new element object

Back to Mini-TOC

---

9. File Identity + Paths (fileLayer.js)

Sketchpad depends on correct file identity and correct path construction. A large class of failures occur when “the UI thinks a file is X” but the filesystem path is actually Y.

9.1 Why fileLayer.js exists

Sketchpad contains multiple content trees:

- /patterns/<category>/
- /gallery/<Domain>/<Category>/
- /home/ (rooted references into other trees)
- /utilities/

fileLayer.js exists to enforce consistent identity rules across these trees.

9.2 Canonical identity rules

Identity rules must define:

- what string is used to match a manifest entry
- what string is used as filename
- what string is used as a script import path

If these do not match, menu commands like archive/edit will fail.

Back to Mini-TOC

---

10. Node Dispatch (nodeLayer.js)

The browser cannot directly perform filesystem operations. Node dispatch provides the bridge.

10.1 Dispatch request structure

A dispatch request contains:

- requestName
- payload

The payload must be shaped by the active tab using uiState context.

10.2 Node dispatch responsibilities

nodeLayer.js must:

- send the request to the service endpoint
- handle HTTP errors as fail-fast

- return structured JSON results

nodeLayer.js must not:

- guess UI meaning
- perform DOM updates
- hide errors

Back to Mini-TOC

---

11. Node Service (sketchpadService.mjs)

sketchpadService.mjs is the central Node service that executes filesystem tasks.

11.1 Service role

The service:

- receives dispatch requests
- executes a named task
- writes files or performs maintenance
- returns a structured result

11.2 Result structure

Results must include enough information for the UI to:

- invalidate caches if needed
- restore the correct view from uiState
- display logs or reports deterministically

Architectural rule: Node service does not “refresh UI.” The UI refreshes itself.

Back to Mini-TOC

---

12. Manifest Maintenance (maintain_manifests.mjs)

maintain_manifests.mjs is the authoritative inventory maintenance script. It ensures that the UI inventory matches the filesystem inventory.

12.1 What it does

- scan directories
- detect categories and items
- generate or update manifest.json files
- validate expected fields
- report added/broken entries

12.2 Why it matters

Without maintenance, Sketchpad can drift into broken states:

- manifest entries pointing to missing files

- files existing on disk but not in manifests
- thumbnails missing for new items
- Home entries referencing stale paths

Architectural rule: tabs do not “repair” inventory; maintenance scripts do.

Back to Mini-TOC

---

13. Per-Tab Ownership Summary

Each tab owns its own regions and behavior. Cross-tab UI modification is not allowed.

13.1 Draw Tab

- inventory: drawRegistry
- rendering: canvas-based objects
- controls: parameterControls driven by controls metadata

13.2 Patterns Tab

- inventory: patterns manifests
- rendering: imported scripts executed on canvas
- navigation: categories + results view + thumbnails

13.3 Gallery Tab

- inventory: per-domain registries + per-category manifests
- rendering: images in #text OR scripts in canvas
- navigation: domain tabs + shared Results tab

13.4 Utilities Tab

- inventory: tools, scripts, reports
- rendering: text reports + canvas scripts
- role: system inspection and maintenance

13.5 Home Tab

- inventory: generated home manifest
- rendering: images or scripts by extension
- role: curated index into other content trees

Back to Mini-TOC

---

14. Diagnostics and Failure Modes (Debug Map)

Sketchpad is fail-fast by design. Missing data is a broken state and must crash loudly.

14.1 Common failure signatures

Examples:

- “Missing manifest.json” → inventory is broken or a path is wrong.

- “Results tab clicked but no saved results context exists” → uiState.saved is inconsistent with the UI.
- “source file not found” → manifest identity mismatch (path/filename mismatch).
- “script module missing expected exports” → script is not conforming to the expected runPattern contract.

14.2 Debugging principle

Because the system is deterministic, the first thrown error is usually the best one. Fix the earliest boundary where state and reality diverged.

Back to Mini-TOC

---

15. Cross-Functional Flows (End-to-End Handoffs)

The subsystem pages explain “what each file owns.” This section explains the end-to-end flows that cut across multiple files. These are the places where responsibility hands off and where most real bugs occur: the UI context must be shaped correctly, identity must be canonical, caches must be invalidated explicitly, and refresh must be driven by uiState restore logic.

> **Canonical template used below**
> Purpose → Trigger → Canonical data object → Sequence → Invariants → Failure signatures → Where to debug first

15.A Caption menu command flow (caption → menu → node → cache → refresh)

Purpose: Execute a command on the currently displayed item (archive/edit/thumbnail/etc.) with correct identity and deterministic UI refresh.

Trigger: User opens the caption menu and selects a command.

Canonical data object: An “info object” built by the active tab that contains:

- domain/category (or tab-specific equivalent)
- manifestPath
- matchField and matchValue (canonical identity)
- filename (canonical file identity string, including extension)
- optional scriptPath (when a script file is the target)
- helpKey/helpSubdirs (for Help integration)

Sequence (handoff chain):

1. Tab computes current selection from uiState and/or tab-local state.
2. Tab builds the info object and passes it into its per-tab menu builder.
3. Per-tab menu builder returns menu items (label + onClick handlers).
4. menuManager renders the menu and invokes the selected handler.
5. Shared menu command calls nodeLayer (nodeDispatch) with requestName + payload.
6. sketchpadService.mjs executes the task (may call maintain_manifests.mjs or file ops).
7. Result returns to browser as structured JSON (and possibly a log name).

8. UI invalidates relevant manifest caches explicitly.
9. UI refreshes by reloading manifest and restoring view from uiState (not “DOM discovery”).

Invariants:

- matchField/matchValue must match the actual manifest entry field and value (no guessing).
- filename must be the canonical identity string (including extension).
- Cache invalidation must happen before any reload/restore step.
- Refresh must be driven by uiState.saved; do not invent a new navigation policy mid-command.

Failure signatures:

- “source file not found” → payload identity/path mismatch (often fileLayer normalization mismatch).
- “manifest entry not found” → matchField/matchValue mismatch or cache not invalidated.
- command appears to work but UI still shows old list → cache invalidation missing or wrong cache key cleared.

Where to debug first: Inspect the built info object (especially matchField/matchValue and manifestPath) and compare it to the manifest entry on disk.

Back to Mini-TOC

15.B Commands offcanvas flow (Commands button → maintenance → refresh)

Purpose: Run a maintenance task (rebuild/validate manifests, etc.) via an offcanvas panel and return to a correct, refreshed view.

Trigger: User clicks the global Commands button, then clicks a command inside the offcanvas.

Canonical data object: The offcanvas command handler uses:

- requestName + payload (often empty for global maintenance tasks)
- the tab’s refresh policy (usually “invalidate cache, reload, restore from uiState.saved”)

Sequence (handoff chain):

1. Tab wires Commands button using uiUtilities.js helpers.
2. showCommandsOffcanvas renders HTML and exposes button IDs.
3. Tab command handler calls nodeRebuildAndValidateManifests (nodeLayer request).
4. Service runs maintain_manifests.mjs (or equivalent) and returns report/log.
5. Tab invalidates the relevant manifest caches explicitly.
6. Tab reloads manifests and restores view from uiState.saved (categories/results/etc.).
7. Tab formats and displays the report in the offcanvas panel.

Invariants:

- Offcanvas HTML IDs must exist (fail-fast if missing).
- Refresh must not overwrite uiState.saved; it must use it.
- Cache invalidation must precede reload.
- Results view restoration must rebuild the same selection and caption.

Failure signatures:

- “#…Button missing” → offcanvas HTML does not define expected IDs.
- After maintenance, Results tab breaks → uiState.saved not preserved or overwritten during refresh.
- Inventory appears unchanged → cache invalidation missing or performed against the wrong cache key.

Where to debug first: Confirm cache invalidation happens, then confirm restore path reads uiState.saved rather than constructing a new default.

Back to Mini-TOC

15.C Manifest identity rules across domains (Patterns vs Gallery vs Home)

Purpose: Ensure that “the string used to identify an item” is consistent across: display, manifest matching, filesystem path resolution, and node tasks.

Trigger: Any action that targets a specific item: archive, rename, edit title, delete, thumbnail generation, “open in offcanvas,” etc.

Canonical data object: A domain-specific identity triple:

- manifestPath (which manifest file contains the entry)
- matchField (which field is authoritative for identity)
- matchValue (the canonical identity string; must match the manifest exactly after normalization rules)

Sequence (handoff chain):

1. UI selects an entry (often containing title/filename/path).
2. Tab converts the entry into the canonical identity triple (via fileLayer rules as needed).
3. Node task uses the identity triple to find the manifest entry and locate the source file.

Invariants:

- matchValue must be the canonical identity string; do not mix “display title” with “file identity.”
- Any normalization (such as stripping redundant category prefix) must be applied consistently everywhere.
- Home is a rooted cross-tree index; its entries must preserve enough path context to resolve back into the correct tree.

Failure signatures:

- “source file not found” → identity string looks correct to the UI but is not the real filesystem path.
- Node reports “entry not found” → matchField/matchValue does not align with the manifest schema for that domain.
- Thumbnail paths look wrong → a second, non-canonical “filename” string is being used for derived assets.

Where to debug first: Inspect the manifest entry on disk and confirm which field is the identity field for that domain. Then confirm the UI's matchValue matches it exactly (after any documented normalization).

Back to Mini-TOC

15.D Tab lifecycle and restore contract (setUI ↔ TabSpec ↔ uiState.saved)

Purpose: Ensure that switching tabs or re-entering a tab always reconstructs the same view deterministically.

Trigger: Tab switch, app reload, or post-maintenance refresh that re-enters a tab.

Canonical data object: The per-tab snapshot: `uiState.<tab>.saved`

Sequence (handoff chain):

1. Leaving tab: nothing is called; the tab has been updating uiState (and, for Home, Patterns and Gallery, uiState.saved) as it goes.
2. Entering tab: setUI decides init vs restore solely from uiState.<tab>.saved existence/meaning.
3. Cold init: tab.init builds UI from defaults and writes uiState.saved.
4. Restore: tab.restore reads uiState.saved and rebuilds view exactly.

Invariants:

- init() must not "fix up" restore behavior.
- restore() must not "assume defaults."
- uiState must be written first, then the DOM is built.
- Tabs own their regions; restore does not reach across into other tabs' UI.

Failure signatures:

- "Results tab clicked but no saved results context exists" → UI allowed a path into restore without a valid uiState.saved.
- Tab comes back in the wrong subtab → uiState.saved is missing activeSubtab, or restore ignores it.
- After maintenance, tab returns to categories unexpectedly → refresh overwrote uiState.saved instead of using it.

Where to debug first: Print uiState.<tab>.saved at the moment restore is invoked and compare it to what the UI is showing.

Back to Mini-TOC

15.E Script execution lifecycle (tab selection → import → runPattern → parameterControls)

Purpose: Execute a script deterministically with consistent UI controls and redraw behavior, regardless of which tab launched it.

Trigger: User selects a script-like item (Patterns entry, Gallery script entry, Utilities script, Home script entry).

Canonical data object: The script module plus the runPattern contract:

- moduleUrl (canonical import path)
- exports: scriptInfo + runPattern (or the expected alternative exports for legacy scripts)
- scriptInfo: title + controls + params + redrawHandler bridge

Sequence (handoff chain):

1. Tab resolves a moduleUrl (using correct identity/path rules).
2. scriptRunner imports the module by moduleUrl.
3. scriptRunner invokes runPattern() as the entry point (canonical path).
4. runPattern prints title, prepares compatibility aliases, calls init(), builds controls, performs first draw.
5. parameterControls calls redrawHandler when controls change.
6. redrawHandler calls update(params) then draw().

Invariants:

- runPattern is the orchestrator and does not contain drawing math.
- draw reads only from elements.element; it does not read params directly.
- update normalizes types and validates; update must not draw.
- The moduleUrl must be correct for the active tab's tree.

Failure signatures:

- "script module missing expected exports" → module does not conform to the required interface.
- Controls change but nothing redraws → redrawHandler missing or not wired to update/draw.
- Initial display is blank → runPattern did not perform an explicit first draw.

Where to debug first: Confirm moduleUrl resolves, then inspect module exports, then confirm redrawHandler is present and calls update/draw.

Back to Mini-TOC

End

This treatise is intentionally long and explicit. It exists so that future changes can be made without re-deriving architecture decisions.

If a new feature requires a structural change, that change should be documented as an explicit architectural update, not introduced silently.

Back to Mini-TOC

Sketchpad Data Flow (Overview)

← Contents

> Checked 27 October 2026. The four loops still describe the app. Two updates: manifests are now cached in `manifest.js` (not in `uiState`), and the Figures tab, which this page predates, adds its own kind of data; see the section added at the end.

This document describes Sketchpad data flow at a conceptual level.

Note: “data flow” in Sketchpad is a broad concept. Sketchpad is not a single pipeline system. It is a collection of cooperating subsystems (tabs, manifests, UI state, drawing runtime objects, overlays, and Node services) that exchange data in predictable ways.

This treatise is therefore an overview, not an exhaustive formal specification.

---

1. The major data domains in Sketchpad

Sketchpad has several distinct kinds of “data,” each with a different lifetime and purpose.

A. UI State (uiState.js)

UI state is the authoritative record of “what the user is looking at right now.”

- active tab
- active category
- active item
- active subtab
- saved view context (categories vs results)
- last selected index for restoration

UI state changes immediately when the user clicks something. Rendering follows state, not the other way around.

B. Manifest Data (directoryRegistry.json + manifest.json)

Manifests are the authoritative inventory of content:

- which categories exist
- which items exist inside each category
- titles, filenames, paths, metadata fields

Manifests are loaded once, cached, and treated as immutable. Tabs only read cached manifest data.

C. Runtime Drawing State (elements / params / controls)

This is the “live” drawing state used while a script or drawRegistry entry is running.

- `params`: current UI values (JSON-safe)
- `controls`: declarative metadata for UI generation

- `elements`: persistent runtime objects used by draw()

Runtime drawing state is created on entry (init) and mutated by update(). The draw() function reads from elements only.

D. Node Service Data (filesystem + generated reports)

Sketchpad includes a Node-side service layer that performs filesystem operations and maintenance tasks. Node tasks produce structured results and log output.

Examples include:

- rebuild and validate manifests
- write thumbnails
- archive scripts or items
- edit manifest titles

Node tasks do not directly manipulate the UI. They return data to the browser, and the browser decides how to display it.

---

2. The four primary data-flow loops

Sketchpad can be understood as four main loops. Each loop is deterministic and has a clear "owner."

Loop 1: UI Navigation Flow (click → uiState → render)

This is the core "UI drives UI" rule.

1. User clicks a tab, category, item, or subtab.
2. The tab controller updates uiState immediately.
3. The tab controller renders the correct view based on uiState.

The key principle is: uiState is always updated first, then rendering happens.

Loop 2: Content Inventory Flow (filesystem → manifest cache → UI)

Manifests define what content exists and how it is organized.

1. On cold start, a tab loads its required manifests.
2. Manifests are cached in memory (manifest.cache).
3. UI rendering reads from the cached data only.

In normal operation, tabs do not repeatedly fetch manifest files. The cache is treated as authoritative until explicitly invalidated.

Loop 3: Drawing Flow (controls → params → update() → elements → draw())

This is the canonical drawRegistry-like lifecycle used by scripts and objects.

1. parameterControls renders UI controls based on controls metadata.
2. Control interaction updates params.
3. redrawHandler calls update(params) to apply changes to elements.element.
4. draw() renders deterministically from elements.element.

The key principle is: draw() does not read params directly.

Loop 4: Maintenance Flow (UI → nodeDispatch → service → UI refresh)

This is the “system maintenance” loop used for tasks like rebuild, archive, edit, and thumbnail generation.

1. User invokes a maintenance command (menu item or offcanvas button).
2. Browser calls nodeDispatch with a request name and payload.
3. Node service performs filesystem work and returns structured results.
4. Browser invalidates caches if needed.
5. Browser restores the tab view from uiState.

The key principle is: Node work changes files, but the browser decides how to reflect that change.

---

3. Data flow by subsystem (high-level map)

Each Sketchpad tab is an independent subsystem. Each subsystem follows the same architectural rules, but reads different data sources.

A. Draw Tab

- Inventory source: drawRegistry (internal JS objects)
- State source: uiState.draw
- Runtime source: drawRegistry entry elements + params

Draw Tab is “object driven” rather than “file driven.”

B. Patterns Tab

- Inventory source: /patterns/<category>/manifest.json
- State source: uiState.patterns
- Runtime source: imported module runPattern()

Patterns is “file driven,” but scripts are expected to behave like drawRegistry objects when executed.

C. Gallery Tab

- Inventory source: /gallery/<Domain>/directoryRegistry.json and per-category manifests
- State source: uiState.gallery
- Runtime source: images or imported script modules

Gallery contains both static content (images) and executable content (scripts).

D. Utilities Tab

- Inventory source: Utilities manifests (and/or hardwired scripts)
- State source: uiState.utilities
- Runtime source: scripts + node service reports

Utilities is the “system workbench.” It is where maintenance and inspection tasks are exposed.

E. Home Tab

- Inventory source: /home/manifest.json (generated by Node)
- State source: uiState.home
- Runtime source: scripts (runPattern) and images

Home is a curated index, not a content storage system. It points into other parts of the filesystem by rooted path.

---

4. What makes Sketchpad “fail-fast”

Sketchpad is intentionally designed to crash loudly when required data is missing. This is not a bug; it is a development strategy.

Examples of fail-fast conditions:

- required DOM regions are missing
- manifest files are missing or unreadable
- uiState saved context is missing or inconsistent
- a script does not export expected fields
- an item path is malformed

Fail-fast behavior makes the data flow visible: the console error shows the exact boundary where state and reality diverged.

---

5. Typical end-to-end flows

Flow A: Select a Pattern and run it

1. User selects Patterns tab.
2. Patterns tab reads cached manifest inventory.
3. User selects a pattern item.
4. uiState.patterns is updated to results view.
5. The module is imported.
6. runPattern() builds controls and draws.

Flow B: Browse Gallery images

1. User selects Gallery tab and a domain (Ideabook / Patterns / Scripts).
2. Gallery reads cached per-domain manifests.
3. User selects an item.
4. uiState.gallery.saved is updated to results view.
5. Caption bar prev/next navigates by updating index and re-rendering.

Flow C: Maintenance task (Rebuild & Validate)

1. User opens Commands offcanvas.
2. User clicks Rebuild & Validate.
3. UI calls nodeDispatch.

4. Node service updates manifests and returns report.
5. UI invalidates cache and restores current view from uiState.
6. UI displays report output.

---

6. What this document does NOT specify

This overview does not attempt to define every file and every function. Sketchpad is intentionally modular and each subsystem owns its own internal details.

Topics not fully specified here include:

- exact manifest schemas per tab
- exact overlayManager layer flow
- caption bar and menu item wiring details
- future persistence (Dexie) for drawRegistry objects
- future figure objects and interconnections

Those are best handled as separate treatises.

Summary

Sketchpad data flow is best understood as multiple deterministic loops:

- UI navigation flow: click → uiState → render
- content inventory flow: filesystem → manifests → cache → UI
- drawing flow: controls → params → update → elements → draw
- maintenance flow: UI → nodeDispatch → service → UI refresh

These loops interact, but they remain separate. This separation is what keeps Sketchpad understandable, testable, and fail-fast.

Added 27 October 2026: Figures

Figures adds a fifth kind of data and a variation on the drawing loop.

E. Figure data (uiState.figures.tabs[tabId].figure)

- Plain data: layers (registry id, parameters, color, visibility, transform), discs, welds, stacking order, active layer. Not saved to disk yet (Phase 10).
- Beside it, not in it: one live instance per layer (`tabState.instances`), a private copy of the registry object made with `Object.create`. A layer's `params` and its instance's `params` are the same object, so a slider or a handle drag changes the figure data directly.

Loop 5: the figure drawing loop

1. A control, handle, disc click or centroid drag changes the figure data (through the shared params, or directly).
2. `redrawFigure()` draws every visible layer bottom to top. While a layer draws, `drawState.transform` holds its transform and `drawLine` maps every line through it.

3. Anchored discs are recomputed from their arm and fraction; then discs and weld marks are drawn.
4. If the change moved welded discs, their partners on other layers are moved the same amount, and the figure is drawn once more.

The rule from Loop 3 still holds inside each layer: the object's `draw()` reads its own elements. What is new is that one canvas holds many objects at once, each in its own coordinates.

How Sketchpad drawRegistry Files Are Organized and Used

← Contents

> Checked 27 October 2026. The lifecycle and the params/elements split described here still hold. The Mystic Rose sample has changed since this page was written: its center is now a draggable handle (`params.points`, with `interactive: true`) instead of a `pointPicker` control (that widget is switched off), and its element is `elements.thing` rather than `elements.element`. Read the sample for its shape, and the current file for its details. What Figures asks of a registry object is added at the end.

This document describes the purpose, structure, and execution flow of a single `drawRegistry` entry file in Sketchpad.

A `drawRegistry` file defines one "first-class drawable object": a reusable, parameter-driven drawing unit that can be selected in the Draw tab, manipulated via controls, and rendered deterministically onto the shared canvas.

The sample file used throughout this discussion is: `drawRegistry/mysticRose.js`

---

1. What a drawRegistry File Is

A drawRegistry file is a JavaScript module that registers a single drawing object on the global `window` namespace using a stable naming convention:

```
window.drawRegistry_mysticRose = { ... };
```

The name `drawRegistry_mysticRose` is not cosmetic: it is the lookup handle Sketchpad uses when building category lists and when instantiating the object at runtime.

Each registry entry is an object with:

- metadata (identity, category, tags, description)
- JSON-safe defaults (`params`)
- UI control descriptors (`controls`)
- a small lifecycle (`init()`, `update()`, `draw()`)
- persistent elements (`elements`) created once and reused

---

2. Sample Registry Entry: Mystic Rose

The example below is a typical drawRegistry file in Sketchpad:

```
/* drawRegistry/mysticRose.js */
import { Point, StringThing }       from '../classes/classes.js';
import { drawMysticRose }           from '../draw/drawUnicorns.js';
```

```
window.drawRegistry_mysticRose = {
    name:         "Mystic Rose (circle)",
    id:           "mysticRose",
    version:      0.1,
    category:     "unicorns",
    firstOrder:   true,
    source:       "internal",
    tags:         ["circular"],
    description:  "Lines drawn to all nodes from each node in a circle",
    status:      "",
    hover:        "",

    // -- visual styling ---
    background: null,
    overlays:   [],
    transforms: [],

    // Placeholder for all elements drawn
    elements:   null,

    // --- Core defaults for drawing (JSON-safe) ---
    params: {
      midpoint: { x: 300, y: 200 },   // converted to Point in init()
      radius:   250,
      numNodes: 20,
      xScale:   1,
      yScale:   1,
      color:    "blue",
      lineWidth: 1
    },

    // --- UI metadata (controls) ---
    controls: {
      radius:    { widget: "range", min: 80,  max: 500, step: 5,   label: "Radius:" },
      numNodes:  { widget: "range", min: 12,  max: 30,  step: 1,   label: "Nodes:" },
      xScale:    { widget: "range", min: 0.5, max: 2,   step: 0.1, label: "X Scale:" },
      yScale:    { widget: "range", min: 0.5, max: 2,   step: 0.1, label: "Y Scale:" },
      color:     { widget: "colorPicker",                          label: "Color:" },
      lineWidth: { widget: "range", min: .5,  max: 3.5, step: .5,  label: "Width:" },
      midpoint:  { widget: "pointPicker",                          label: "Midpoint:" }
    },

    init() { ... },
    update(params) { ... },
    draw() { ... }
};
```

The remainder of this treatise explains how Sketchpad expects each section to behave.

---

3. Identity and Metadata Fields

A drawRegistry entry begins with identity and descriptive metadata. These fields are used by the Draw tab UI and by future persistence and search features.

3.1 Display and Identity

- `name`
  Human-readable display name used in category lists and captions.
- `id`
  A stable internal identifier for the object type. This is intended to remain constant even if the display name changes.
- `version`
  A numeric version for the registry entry definition. This can be used later for migration of saved objects.
- `category`
  The Draw tab category the entry belongs to. This is the primary grouping key.

3.2 Classification / System Fields

- `firstOrder`
  A flag indicating that this is a first-class object intended for direct use in Draw and later Figures. (This is conceptual and UI-driven.)
- `source`
  Indicates where the object definition comes from (example: internal).
- `tags`
  A small array of search keywords. Example: `["circular"]`.
- `description`
  A one-sentence description of what the object draws.

3.3 Status and Hover (Optional Display Fields)

- `status`
  A short status string (often blank). This can be used later for filtering, “new/working/finished” marking, etc.
- `hover`
  A short hover hint string. This can be used later in UI overlays.

---

4. Styling / Rendering Policy Fields

Many drawRegistry entries include placeholders for rendering policy:

- `background` — optional background fill or configuration
- `overlays` — optional overlay layers to draw or enable
- `transforms` — optional transforms applied to the drawing

In the example:

```
 background: null,
overlays:   [],
transforms: [],
```

These are present so the registry entry can eventually become a fully self-contained “drawing recipe,” not just a geometry generator. In early versions they often remain inert.

---

5. The Two Kinds of State: params vs elements

The most important conceptual split in a drawRegistry entry is:

- `params` = JSON-safe values that represent the editable “inputs” to the drawing.
- `elements` = persistent runtime objects created once, stored on the entry, and mutated as the drawing changes.

This split is deliberate and supports:

- persistence (save/load params as JSON)
- interactive editing (controls modify params)
- stable objects (avoid recreating geometry every redraw)
- deterministic rendering (draw reads from a known element graph)

5.1 params: JSON-safe defaults

`params` must be safe to serialize as JSON. This means: no class instances, no functions, no DOM nodes.

Example:

```
params: {
  midpoint: { x: 300, y: 200 },
  radius:   250,
  numNodes: 20,
  xScale:   1,
  yScale:   1,
  color:    "blue",
  lineWidth: 1
},
```

Note that `midpoint` is stored as a plain object with `x/y` rather than a `Point` instance.

5.2 elements: persistent objects created in init()

`elements` is the container for persistent runtime objects. In the example, init() creates one element:

```
this.elements = { element: new StringThing(this.params) };
```

This is the “live object” that draw() will render.

---

6. controls: UI Metadata for parameterControls

The `controls` object describes how each parameter should be edited. It is a declarative map keyed by parameter name:

```
controls: {
  radius:    { widget: "range", min: 80,  max: 500, step: 5,   label: "Radius:" },
```

```
  numNodes:  { widget: "range", min: 12,  max: 30,  step: 1,   label: "Nodes:" },
  xScale:    { widget: "range", min: 0.5, max: 2,   step: 0.1, label: "X Scale:" },
  yScale:    { widget: "range", min: 0.5, max: 2,   step: 0.1, label: "Y Scale:" },
  color:     { widget: "colorPicker",                          label: "Color:" },
  lineWidth: { widget: "range", min: .5,  max: 3.5, step: .5,  label: "Width:" },
  midpoint:  { widget: "pointPicker",                          label: "Midpoint:" }
},
```

The Draw tab reads this metadata and builds the control panel dynamically.

6.1 Widget types in the example

- `range`
  Numeric slider with min/max/step.
- `colorPicker`
  A color selector that returns a string (example: "blue" or "#0000ff").
- `pointPicker`
  A specialized control that allows selecting a point on the canvas (or entering coordinates) and returns either a Point-like object or a `Point` instance depending on implementation.

The key principle is that `controls` describes UI behavior, while `params` contains the current editable values.

---

7. The Lifecycle: init() → update() → draw()

Every drawRegistry entry implements three core functions:

- `init()` — cold-start construction of persistent elements
- `update(params)` — apply new values to the persistent elements
- `draw()` — render the current element state onto the canvas

This lifecycle makes the object deterministic and repeatable: the same params produce the same drawing, and update() does not depend on hidden external state.

7.1 init(): build persistent geometry and style

init() runs once when the entry becomes active for the first time. Its job is to convert JSON-safe params into concrete objects.

```
init() {
  // Ensure midpoint is a Point object
  const p = this.params.midpoint;
  if (!(p instanceof Point)) this.params.midpoint = new Point(p.x, p.y);

  this.elements = { element: new StringThing(this.params) };
}, // end init
```

Two important things happen here:

- The midpoint is converted from a plain `{x,y}` into a real `Point`.
- The registry entry creates a persistent `StringThing` that will be reused.

7.2 update(params): apply changes from UI controls

update(params) receives a new parameter object from the UI system. It must update the persistent element graph without rebuilding it.

```
update(params) {
  const e = this.elements.element;
  for (const key in this.params) {
    const value = params[key];
    if (value === undefined) continue;

    if (key === "midpoint") {
      if (value instanceof Point)
        e.midpoint = value;
      else
        e.midpoint = new Point(value.x, value.y);
    } else if (key == "radius") {
      e.ellipse.a = value;
      e.ellipse.b = value;
    } else {
      e[key] = value;
    }
  }
}, // end update
```

This update() illustrates a common pattern:

- loop over known params (not over user input keys)
- apply only defined values
- handle special cases explicitly (midpoint conversion, radius mapping)
- otherwise assign directly

The radius logic shows that the underlying geometry may not map 1:1 to params:

```
} else if (key == "radius") {
  e.ellipse.a = value;
  e.ellipse.b = value;
}
```

This indicates that `StringThing` internally uses an ellipse model even when the UI is conceptually “radius”.

7.3 draw(): render deterministically

draw() is the simplest function: it renders the current element state.

```
draw() {
  const thing = this.elements.element;
  drawMysticRose(thing);
} // end draw
```

draw() does not rebuild geometry, does not read from UI controls, and does not attempt to “fix” missing state. It assumes init() and update() have produced a valid object graph.

---

8. How Sketchpad Uses a drawRegistry Entry at Runtime

The typical execution flow when the user selects an entry in the Draw tab is:

1. Draw tab selects the registry entry object (by category + id)
2. Draw tab calls `entry.init()` once (cold-start)
3. Draw tab builds parameter controls from `entry.controls`
4. When a control changes, Draw tab calls `entry.update(newParams)`
5. Draw tab calls `entry.draw()` after update

This yields a consistent model:

- controls drive params
- params drive elements
- elements drive drawing

---

9. Naming Conventions and File Organization

In Sketchpad, drawRegistry entries typically live under:

```
./drawRegistry/
```

A file name generally corresponds to the registry name suffix:

```
drawRegistry/mysticRose.js
window.drawRegistry_mysticRose
```

This convention keeps the system predictable and easy to debug: when something fails, you can locate the registry entry by name immediately.

---

10. Practical Debugging Notes

10.1 Fail-fast behavior

The drawRegistry model works best when errors are allowed to surface immediately. If a required element is missing, it should throw rather than silently degrade.

10.2 Common failure points

- **Missing import** for a geometry class or draw function
- **params not JSON-safe** (class instances embedded in defaults)
- **init() not called** before update/draw
- **update() mismatched keys** (UI provides keys that are not handled)
- **draw() assumes elements exist** but init() never created them

---

11. Summary: The drawRegistry Contract

A drawRegistry file defines a single drawing object with:

- stable identity: `id`, `category`, `name`
- JSON-safe defaults: `params`
- UI metadata: `controls`
- persistent runtime elements: `elements`
- deterministic lifecycle: `init()` → `update()` → `draw()`

This structure supports incremental development: you can begin with one line, then add more geometry and more parameters, while keeping the same lifecycle and mental model.

Added 27 October 2026: what Figures asks of a registry object

Any registry object can be inserted into a figure. These extra rules make it work well there:

- **Keep all state on `this.`** Each layer is `Object.create(entry)` with its own `params` and `elements`. State kept anywhere else (a module-level variable, the entry object itself) would be shared by every copy, so two Parabolas would fight over it.
- **Draw through `drawLine`** (`draw/drawUtilities.js`). That is how a layer's move and turn reach the object without it knowing: `drawLine` maps both ends through the layer's transform and records the extent for the centroid. Direct `ctx` drawing would not move with the layer.
- **Handles in `params.points`**, with `interactive: true`. The interactor drags these. The object's `update()` reads them back (and decides what a moved midpoint means).
- **`infrastructure()`** (optional): returns the object's arms, the lines discs can sit on, as `{ id, a, b, steps, from, to }` or `{ id, points, from, to }`. Without it, only free discs can be placed on the object. Nine of the sixteen registry objects have one.
- **`decompose()`** (optional): returns `[{ key, params }]`, the pieces for Break apart. Only Regular Polygon has one so far.

runPattern() in Sketchpad

← Contents

> Checked 27 October 2026. The contract stands. The loader side has moved on: tabs now call `runScriptByPath()` in `scriptRunner.js`, which stops the previous script (if it exports `stop()`), puts the canvas in place, fills it white, calls `runPattern()` with no argument, then arms the handle dragger for an interactive `scriptInfo` and builds its controls if the tab asked for them. See the scriptRunner page.

This document explains the role, contract, and expected behavior of `runPattern()` as used throughout Sketchpad.

In Sketchpad, `runPattern()` is the canonical entry point for running a drawing script. It is not a general-purpose "draw function." It is an orchestrator that:

- initializes persistent runtime state (elements)
- connects UI controls (parameterControls) to that state
- triggers a deterministic first render

Where runPattern() is used

Sketchpad executes scripts from multiple places (Patterns, Gallery Scripts, Utilities Lab, and Home). Regardless of where the script originates, the execution flow is the same: the loader imports the module, then calls `runPattern()`.

The loader (for example, `scriptRunner.js` or a tab controller) is responsible for:

- attaching the shared canvas to the correct region (`#sketchpad`)
- clearing the canvas to a known background (typically white)
- calling `runPattern()` with the expected signature for that subsystem

The script itself is responsible for everything else.

runPattern() is the orchestrator

`runPattern()` exists to enforce a single predictable lifecycle:

1. present title (human context)
2. create runtime elements (persistent objects)
3. build UI controls (parameterControls)
4. perform first draw (explicit)

The key design principle is separation of responsibilities:

- `init()` creates objects
- `update(params)` applies new values
- `draw()` renders deterministically
- `runPattern()` ties them together in the correct order

Canonical script model used by runPattern()

Sketchpad prefers scripts that follow the same vocabulary as drawRegistry entries, even when they are not actually drawRegistry items.

A compatible module typically contains:

- `scriptInfo` (metadata + UI contract)
- `params` (JSON-safe defaults + current UI values)
- `controls` (parameterControls schema)
- `elements` (persistent runtime state)
- `init()` (cold-start only)
- `update(newParams)` (apply new values, do not draw)
- `draw()` (render only, do not read params)
- `runPattern()` (orchestrator)

runPattern() required sequence

A canonical `runPattern()` must perform these steps in order:

1. `printTitle(scriptInfo.title)`
   The script title is printed to the UI (or overlay) so the user knows what is running.
2. Compatibility alias setup
   parameterControls currently expects `scriptInfo.parameters`. If the script uses `scriptInfo.params`, it must alias:

   ```
   scriptInfo.parameters = scriptInfo.params
   ```

3. `init()`
   Called exactly once per script load. This creates `elements` and seeds `elements.element` from `params`.
4. `buildParameterControls(scriptInfo, targetId, true)`
   This builds the controls UI from `scriptInfo.controls` and binds user changes to the script's redraw behavior.

   The targetId is owned by the Sketchpad UI layout and must not be changed by the script.
5. First draw (explicit)
   A correct script must always do an explicit first render. The typical pattern is:

   ```
   scriptInfo.redrawHandler();
   ```

ParameterControls compatibility bridge

Sketchpad's parameterControls expects a specific interface on `scriptInfo`. To keep conversions mechanical and low-risk, scripts provide a bridge.

The required bridge fields are:

- `scriptInfo.title`
- `scriptInfo.controls`
- `scriptInfo.parameters` (alias to params)
- `scriptInfo.redrawHandler`
- `scriptInfo.onParamChange` (may be no-op)

The canonical redraw handler is:

```
scriptInfo.redrawHandler = function () {
  update(this.params);
  draw();
};
```

This preserves the required separation:

- controls mutate params
- update copies params into elements.element
- draw reads elements.element and renders

What runPattern() must NOT do

A correct `runPattern()` must never:

- create or resolve `ctx` (ctx is global and must already exist)
- accept a `ctx` parameter
- cache or alias `ctx`
- do drawing logic itself (drawing belongs in `draw()`)
- mutate `controls`

Common failure modes

Controls move but nothing redraws

Cause: missing or broken `scriptInfo.redrawHandler`.

Drawing ignores control values

Cause: `draw()` is reading from `params` or stale locals instead of reading from `elements.element`.

Blank initial render

Cause: `runPattern()` did not do an explicit first draw.

Wrong numeric behavior

Cause: type normalization missing in `update()` (range controls deliver strings unless converted).

Minimal canonical runPattern() example

This is a minimal structural example. It shows the required ordering and the compatibility bridge. The math/drawing inside `draw()` is intentionally omitted.

```
export const scriptInfo = {
  title: "Example Script",
  params: {
    radius: 100
  },
  controls: {
    radius: { widget: "range", min: 10, max: 300, step: 1, label: "Radius:" }
  }
};

export let elements = null;

export function init() {
  elements = { element: { radius: scriptInfo.params.radius } };
}

export function update(newParams) {
  elements.element.radius = Number(newParams.radius);
}

export function draw() {
  // draw using ctx and elements.element only
}

export async function runPattern() {
  printTitle(scriptInfo.title);

  // Compatibility alias for parameterControls
  scriptInfo.parameters = scriptInfo.params;

  scriptInfo.redrawHandler = function () {
    update(this.params);
    draw();
  };

  scriptInfo.onParamChange = function () {
    // optional no-op; may be required by parameterControls
  };

  init();

  buildParameterControls(scriptInfo, "action", true);

  scriptInfo.redrawHandler();
}
```

Summary

In Sketchpad, `runPattern()` is the single canonical entry point for running a script. It exists to enforce a deterministic lifecycle and keep drawing scripts structurally consistent with drawRegistry.

If a script follows the init/update/draw model and provides the parameterControls compatibility bridge, it will run predictably in Patterns, Gallery Scripts, Utilities Lab, and Home.

Design Rules: Gallery Script Conversion (Authoritative)

← Contents

> Checked 27 October 2026. The contract stands as written. One thing has changed around it: `scriptRunner.js` now also builds a script's controls after `runPattern()` returns, when the tab asks for controls and the script exports `scriptInfo`. A converted script that builds its own controls in `runPattern()` (as step 4 below requires) therefore has them built twice, with the same result. Harmless; noted so it is not mistaken for a bug.

This document is the authoritative conversion contract for converting standalone drawing scripts into Gallery scripts.

It is meant to be pasted back verbatim before any future conversion request. No interpretation. No optimization. No “helpful” adjustments.

FOUNDATIONAL CONSTRAINTS (NON-NEGOTIABLE)

1. There is exactly ONE canvas and ONE drawing context (ctx).
2. ctx is predefined, global, and authoritative.
3. All drawing code assumes ctx already exists.
4. No script may create, cache, pass, alias, or resolve ctx.
5. No script may reference window.ctx.
6. If ctx does not exist, the system must crash loudly.
7. Conversions MUST NOT introduce any ctx-related logic.

This rule exists because:

- The entire Sketchpad system is built on a single shared coordinate space.
- Overlays, nodes, handles, guides, and figures must interconnect exactly.
- External standalone drawings must be drop-in compatible with Sketchpad.
- Any ctx indirection risks breaking that guarantee.

All draw code uses ctx directly. Period.

PURPOSE OF CONVERSION

The conversion exists for ONE reason only: To make Gallery scripts look, read, and behave like drawRegistry entries, so that:

1. You can mentally read any Gallery script the same way you read a drawRegistry entry.
2. parameterControls can treat Gallery scripts and drawRegistry entries consistently.
3. The conversion is mechanical, repeatable, and low-risk.

This is NOT:

- A drawRegistry conversion

- A refactor
- An architectural redesign
- A math rewrite

CANONICAL INTERNAL MODEL (MANDATORY)

Every converted Gallery script MUST implement the following internal vocabulary and lifecycle:

1. params

- JSON-safe defaults.
- Holds current UI values.
- Replaces any previous parameters object.
- Must NOT be used directly for drawing.

2. controls

- Declarative metadata for parameterControls.
- Schema must remain exactly as expected:
  - label
  - widget
  - min / max / step
  - options
  - default
- Must never be mutated.

3. elements

- Persistent runtime state.
- Created exactly once in init().
- Must have the shape:

```
elements = { element: { … } }
```

- elements.element is the only object draw() reads from.
- May contain:
  - normalized values
  - derived values
  - cached geometry
  - arrays, points, etc.

4. init()

- Cold-start only.
- Creates elements.
- Seeds elements.element from params.
- May validate initial values.

- May compute derived values.
- MUST NOT:
  - build UI
  - draw
  - reference parameterControls
  - touch ctx beyond assuming it exists

5. update(newParams)

- Copies values from params into elements.element.
- Handles all type normalization (parseFloat, parseInt, booleans).
- Handles all validation (fail-fast).
- Recomputes derived data.
- MUST NOT draw.
- MUST NOT create new element objects.
- MUST NOT mutate controls.

6. draw()

- Reads ONLY from elements.element.
- Clears and draws deterministically.
- Uses ctx directly.
- MUST NOT read from params.
- MUST NOT mutate params or controls.

---

runPattern() — GALLERY ENTRY POINT

runPattern() is the orchestrator.

It MUST perform the following steps in order:

1. printTitle(scriptInfo.title)
2. Compatibility alias setup

   ```
   scriptInfo.parameters = scriptInfo.params
   ```

3. init()
   Called exactly once per script load.
4. buildParameterControls(scriptInfo, targetId, true)
   - targetId must not change.
5. First draw
   - Must be explicit.
   - Typically via scriptInfo.redrawHandler().

No ctx handling of any kind occurs here.

PARAMETERCONTROLS COMPATIBILITY (MANDATORY BRIDGE)

parameterControls currently expects these fields:

- scriptInfo.title
- scriptInfo.controls
- scriptInfo.parameters
- scriptInfo.redrawHandler
- scriptInfo.onParamChange (may be required even if empty)

Because the canonical model uses params/init/update/draw, a compatibility bridge is REQUIRED.

The bridge is:

1. scriptInfo.parameters

- Alias to scriptInfo.params.

2. scriptInfo.redrawHandler

- Must exist.
- Must do exactly:

```
update(this.params);
draw();
```

3. scriptInfo.onParamChange

- Must exist.
- May be a no-op.
- May alias redrawHandler if required.

This allows:

- Immediate standardization
- Zero changes to parameterControls
- Safe batch conversion of many scripts

MECHANICAL CONVERSION CHECKLIST

For each script, do ONLY the following:

1. Leave all math and drawing logic untouched.
2. Rename parameters → params.
3. Introduce elements and init().
4. Implement update(params).
5. Implement draw().
6. Add compatibility aliases.
7. Update runPattern() to follow the required sequence.

Nothing else.

WHAT MUST NEVER CHANGE

During conversion, DO NOT:

- Touch ctx handling.
- Add ctx parameters.
- Add ctx caching.
- Add safety wrappers for ctx.
- Change geometry math.
- Change coordinate systems.
- Change colors or defaults.
- Change canvas clearing behavior.
- Change buildParameterControls usage.
- Introduce new abstractions.

FAILURE MODES (DIAGNOSTIC MAP)

1. Controls move, nothing redraws
   → Missing or broken redrawHandler.
2. Drawing ignores control values
   → draw() reading params or stale locals instead of elements.element.
3. Blank initial render
   → No explicit first draw in runPattern().
4. Wrong numeric behavior
   → Type normalization missing in update().
5. Extra testing burden
   → Conversion touched things it was not allowed to touch.

AUTHORITATIVE SUMMARY

A converted Gallery script:

- Uses drawRegistry vocabulary.
- Preserves Gallery execution semantics.
- Assumes ctx exists and uses it directly.
- Contains NO ctx logic whatsoever.
- Is structurally uniform with every other converted script.
- Requires minimal testing because nothing else changed.

Coding Simplicity

← Contents

> Checked 27 October 2026. A statement of principle; nothing in it depends on code that has changed. It still applies, and the Figures work (Phases 5 to 8) was written to it.

This document describes a philosophy of simplicity for code written in Sketchpad, derived from working sessions between Barry and Claude. It exists because Claude's trained instincts tend toward complexity — frameworks, derived quantities, general machinery — and this document is intended to be read at the start of a session to orient Claude toward a simpler way of working.

The core principle is: **code should reflect the idea directly, not the machinery needed to produce the output.**

Part 1 — The Central Pattern

Almost every simplification in practice comes down to one distinction:

- **Claude's instinct:** compute a value from a formula.
- **Barry's instinct:** start at a point and increment.

When seeding points on a grid, Claude reaches for:

```
x = xLeft + (xRight - xLeft) * i / (M - 1)
```

Barry replaces this with:

```
for (let x = x0; pts.length < M; x += dx) pts.push(x);
```

The formula computes each position independently from derived bounds. The increment just adds a delta to the previous value. The increment version is shorter, more readable, and makes the structure of the thing visible at a glance. It also makes parameters obvious: the only things that matter are the starting point and the delta — not a left bound, a right bound, and a count all held together.

> **Rule:** When building a sequence of evenly spaced values, always choose increment-from-start over formula-from-bounds. The starting point and the delta are the natural parameters. Everything else is derived and should not appear in the code.

Part 2 — Don't Build the General Machinery

Claude's instinct when faced with a pattern is to build the general case first. Barry's instinct is to name the actual patterns directly.

Example: Sliding window vs. named triplet types

When building parabola triplets from a grid of points, Claude's approach was to build an intermediate sequence array and slide a window of 3 across it:

```
const seq = [];
for (let j = 0; j < M; j++) {
  seq.push(pt(extRow, j));
  if (j < M - 1) seq.push(pt(bandRow, j));
}
for (let k = 0; k < seq.length - 2; k++)
  triplets.push([seq[k], seq[k+1], seq[k+2]]);
```

Barry recognized that the sliding window only ever produces two shapes:

- **Type A:** `[ext[j], band[j], ext[j+1]]` — apex on band row
- **Type B:** `[band[j], ext[j+1], band[j+1]]` — apex on exterior row

The simpler version names these directly:

```
/* Type A: apex on band row */
for (let j = 0; j < M - 1; j++)
  triplets.push([pt(extRow, j), pt(bandRow, j), pt(extRow, j+1)]);

/* Type B: apex on exterior row */
for (let j = 0; j < M - 2; j++)
  triplets.push([pt(bandRow, j), pt(extRow, j+1), pt(bandRow, j+1)]);
```

No intermediate array. No window. The geometry is visible in the code.

> **Rule:** Before building general machinery (sequences, windows, indices), ask: what are the actual things being produced? Name them directly. The general machinery is often hiding two or three simple cases that can be stated plainly.

Part 3 — Don't Compute What You Can Choose

A related instinct in Claude is to derive parameters from other parameters. Barry's approach is to choose the primitive directly and let everything else follow.

Example: Grid layout

When seeding a grid of points, Claude computed the grid dimensions from `radius`, `midpointX`, and `midpointY`:

```
const xLeft  = cx - r / 2;
const xRight = cx + r / 2;
const topX = Array.from({ length: M }, (_, i) =>
  xLeft + (xRight - xLeft) * i / (M - 1)
);
```

Barry replaced this with: choose a top-left corner and a delta.

```
const x0 = 100, y0 = 50;
const dx = params.width * params.scale;
```

```
const dy = params.height * params.scale;
```

Everything else falls out from these four values. There is no derived left bound, right bound, or center. The consequence is that adding width, height, and scale controls is a trivial change — you just expose the primitives that were already there. In the original version, it would have required rethinking the entire coordinate derivation.

> **Rule:** Expose primitives, not derived quantities. If a parameter is computed from two others, ask whether those two others are the more natural controls. Parameters should be the things you actually want to adjust, not the things the formula needs.

Part 4 — Don't Create Points You Don't Use

Claude's first grid implementation stored a full `(2N-1) × (2M-1)` rectangular grid — every intersection of exterior and band rows and columns. Only half of those points were ever referenced in the parabola triplets. The other half appeared as ghost drag handles with no geometry attached.

The fix was to recognize that exterior rows and band rows have different x sequences:

- Exterior rows: `x0, x0+dx, x0+2dx, ...` (M values)
- Band rows: `x0+dx/2, x0+3dx/2, ...` (M-1 values, the midpoints)

Once that structure was explicit, `_seed` became:

```
for (let i = 0, y = y0; i < 2*N-1; i++, y += dy) {
  const xs = (i % 2 === 0) ? extX : bandX;
  xs.forEach(x => params.points.push({ x, y }));
}
```

Every point stored is a point used. No orphans.

> **Rule:** Only create what is needed. If points, intermediate arrays, or objects are created but not all used, that is a sign the structure does not match the actual geometry. Find the structure that matches, and the unused elements disappear.

Part 5 — The Working Method

The simplifications above do not happen automatically. They require a particular way of working together.

Think before writing

Claude should describe its intended approach before writing any code. Barry will often stop it here and redirect toward the simpler version. This is more efficient than writing code and then simplifying it.

Ask: what is it really?

When Claude proposes a structure, Barry often responds with "not quite" and restates the thing more directly. The correct response to this is not to add another layer of abstraction — it is to ask: what is the actual thing being described? Usually the answer is smaller and more direct than what Claude proposed.

Small steps, verified

Each change is small and verified before the next one is made. This is not just about coding comfort — it preserves architectural ownership. When each step is understood before the next begins, the person writing the code understands every line of it.

Eliminate, don't guard

When a block of code is wrong, remove it entirely. Do not add a conditional guard around it. Guarded-but-wrong code accumulates and obscures the structure.

> **Rule:** The goal is code where the idea is visible. If you have to explain why a block of code is there, the code is probably not simple enough yet.

Part 6 — Before/After Reference

The following pairs show Claude's instinct alongside the simpler replacement. When Claude notices it is about to write something in the left column, it should stop and find the right column version instead.

Claude's instinct	Simpler replacement
Compute position from formula using derived bounds	Start at a point, increment by delta
Build intermediate sequence array, slide window of 3	Name the two triplet types, push directly
Store full rectangular grid of all intersections	Store only the points that are actually used
Expose radius, midpointX, midpointY as controls	Expose width, height, scale — the actual primitives
Add a conditional guard around wrong code	Remove the wrong code entirely
Build the general case first	Name the actual cases that exist and state them plainly
Write code, then simplify	Describe approach first, think before writing

---

Summary

Simplicity in this codebase is not a style preference — it is a structural property. Code that reflects the idea directly is easier to change, easier to debug, and easier to extend. The joinedParabs example demonstrates this concretely: adding width, height, and scale controls to

the final version was a three-line change because the primitives were already explicit. The same change to the original version would have required rethinking the entire coordinate system.

The discipline required is: stop before building, ask what the thing really is, and find the version where the answer is visible in the code.

Undo in Figures: a Proposal

NEW

← Contents

> Written 27 October 2026 for discussion. Nothing here is built. Barry asked where undo came from: it came up while specifying the next weld pass (an accidental handle drop would weld, and there is no way back except right-click). Barry also recalls an earlier remark that undo could be done by taking snapshots of a portion of `uiState`. This page works that idea out.

The idea in one paragraph

Everything that makes a figure what it is lives in one plain-data object, `uiState.figures.tabs[tabId].figure`: its layers (each with its object's parameters, color, visibility and transform), its discs, its welds, the stacking order, the active layer. Plain data can be copied whole with `structuredClone()`. So: before each change, push a copy onto a stack. Undo pops the copy, puts it back, and rebuilds the live drawing objects from it. Nothing has to know how to reverse its own change; there is no list of inverse operations to keep correct.

Why it is cheap here

- A figure is small: a few kilobytes even with dozens of layers and discs. Fifty copies is nothing.
- It was kept plain data on purpose (see `figureModel.js`), for Save As. Undo gets the same benefit.
- Welds, discs and layers are all in the one object, so a snapshot can never restore a weld without the discs it refers to.

The one real piece of work: rebuilding the instances

Each layer has a live instance (a private copy of its registry object) in `tabState.instances`, and today `layer.params` and `instance.params` are the same object. After a snapshot is put back, those links are broken: the restored `layer.params` is a copy. So every instance has to be made again from `layer.id` and `layer.params`, with its callbacks (`redrawHandler`, `getTransform`, `yieldTo`, `beforeMove`, `afterMove`) attached.

`insertLayer()` already does exactly this for one layer, mixed in with "add a new layer". The proposal is to pull out a `makeInstance(tabId, index)` that both use. **Phase 10 (reopening a saved figure) needs the same function**, which is the strongest reason to build undo next to it: one rebuild path, written once, tested twice.

When to take a snapshot

Once per action, before it starts, never on every mouse move:

Action	Snapshot taken
Drag a handle	on the press, not each move (the first `beforeMove` of a drag)
Move or turn a layer	on the press on the diamond or the rotation handle

Arrow-key nudge	once per run of presses (a pause starts a new step)
Insert, Break apart	before
Place, delete or drag a disc	before
Weld, unweld	before
Show or hide a layer, change its color	before
Move a slider	when the slider is let go (its `change` event), not while it slides

A new action after an undo clears the redo stack, as usual.

What is not undone

View state, which is not part of the figure: Points Mode, Show All, the selected disc or handle, whether the centroid is selected. Undo leaves those as they are (and clears the selections, since the thing selected may be gone).

How it would look

- Ctrl+Z undo, Ctrl+Y (and Ctrl+Shift+Z) redo, when the keyboard is not in a text box or slider.
- Undo and Redo in the caption menu, grayed when there is nothing to undo or redo.
- One stack per figure subtab, kept in its `tabState` (view state, not saved with the figure).

Where the code would go

- A new `ui/figures/figureUndo.js`: `snapshot(tabId)`, `undo(tabId)`, `redo(tabId)`, the stacks and their limit.
- `figureLayers.js`: `makeInstance()`, and a `rebuildInstances(tabId)` that undo and Phase 10 call.
- One `snapshot()` call at the start of each action listed above (about a dozen places).

Questions for Barry

1. How many steps back? (Proposed: 50.)
2. Is a slider move an undo step? (Proposed: yes, one per release.)
3. Build it with Phase 10, sharing the rebuild code, or sooner on its own? (Proposed: with Phase 10.)
4. Should undo also cover closing a figure subtab by mistake? That would need the stack to outlive the subtab; it is a separate, smaller "reopen closed figure" feature.